

Comprehensive Architecture and Codebase Documentation for Online Learning Portal

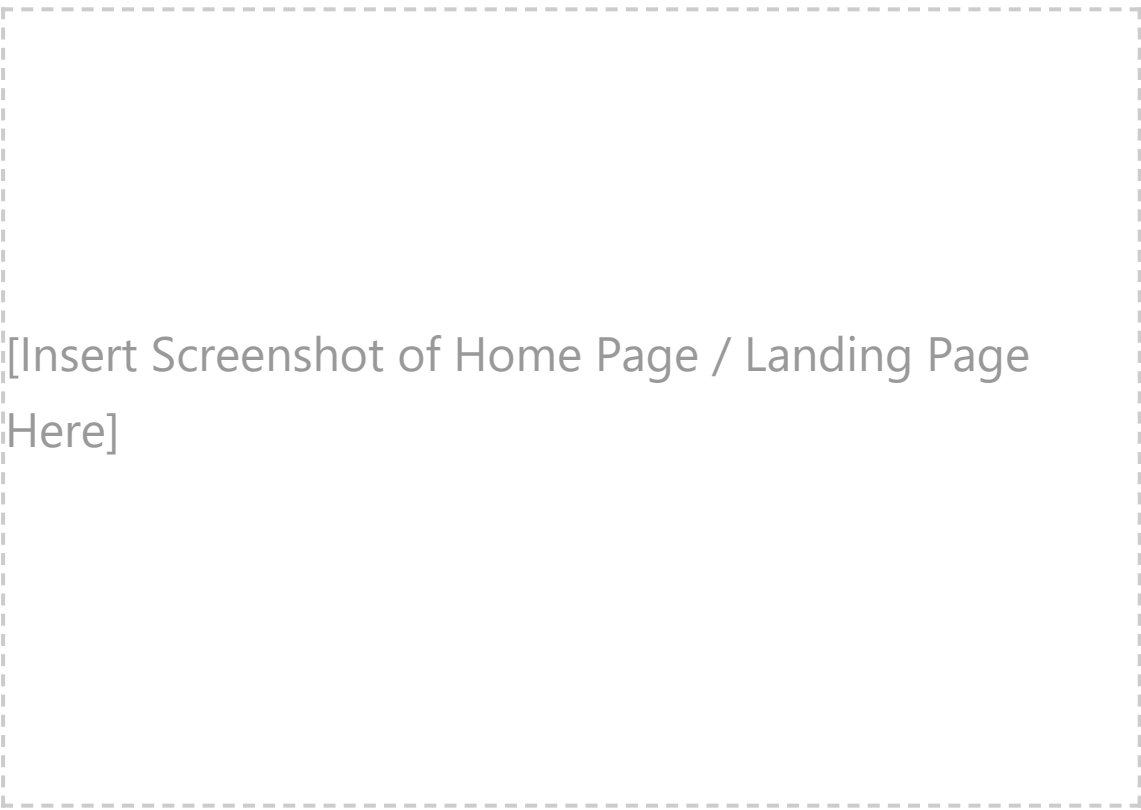
1. System Architecture Overview

The application is a typical **MERN** stack application featuring strict role-based access control.

- **Frontend:** Built with **React** (TypeScript, Vite), styled with **TailwindCSS**, uses `react-router-dom` for navigation. Global state via `AuthContext` .
- **Backend:** Powered by **Node.js** and **Express.js**. Exposes a RESTful API and utilizes **Mongoose** for MongoDB. Handles complex file storage with Multer and GridFS.

2. User Interfaces (Image Placeholders)

2.1 Home Page / Landing Page



2.2 User Registration Page



[Insert Screenshot of User Registration Page Here]

2.3 User Login Page

[Insert Screenshot of User Login Page Here]

2.4 Admin Dashboard - Overview

[Insert Screenshot of Admin Dashboard - Overview Here]

2.5 Admin Dashboard - User Approval List

[Insert Screenshot of Admin Dashboard - User Approval List Here]

2.6 Admin Dashboard - Site Settings (Branding)

[Insert Screenshot of Admin Dashboard - Site Settings (Branding) Here]

2.7 Teacher Dashboard - Courses Overview

[Insert Screenshot of Teacher Dashboard - Courses Overview Here]

2.8 Teacher Dashboard - Create/Edit Course

[Insert Screenshot of Teacher Dashboard -
Create/Edit Course Here]

2.9 Teacher Dashboard - Upload Material / Create Lesson

[Insert Screenshot of Teacher Dashboard - Upload
Material / Create Lesson Here]

2.10 Teacher Dashboard - Quiz Creation

[Insert Screenshot of Teacher Dashboard - Quiz
Creation Here]

2.11 Student Dashboard - Enrolled Courses

[Insert Screenshot of Student Dashboard - Enrolled
Courses Here]

2.12 Student Dashboard - Course Catalog

[Insert Screenshot of Student Dashboard - Course Catalog Here]

2.13 Student Dashboard - Taking a Quiz

[Insert Screenshot of Student Dashboard - Taking a Quiz Here]

2.14 Student Dashboard - Downloading Materials

[Insert Screenshot of Student Dashboard -
Downloading Materials Here]

3. Frontend Source Code (React / TypeScript)

3.1 Root

File: `client/src/App.tsx`

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Home from './pages/Home';
import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';
import CreateCourse from './pages/CreateCourse';
import ManageCourse from './pages/ManageCourse';
import CreateQuiz from './pages/CreateQuiz';
import TakeQuiz from './pages/TakeQuiz';
import CourseList from './pages/CourseList';
import CourseDetail from './pages/CourseDetail';
import { AuthProvider } from './context/AuthContext';

const App: React.FC = () => {
  return (
    <Router>
```

```

    <AuthProvider>
      <div className="min-h-screen bg-gray-50 text-gray-900 font-sans">
        <Navbar />
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/create-course" element={<CreateCourse />} />
          <Route path="/manage-course/:id" element={<ManageCourse />} />
          <Route path="/create-quiz/:courseId" element={<CreateQuiz />} />
          <Route path="/take-quiz/:id" element={<TakeQuiz />} />
          <Route path="/courses" element={<CourseList />} />
          <Route path="/courses/:id" element={<CourseDetail />} />
        </Routes>
      </div>
    </AuthProvider>
  </Router>
);
};

export default App;

```

File: client/src/main.tsx

```

import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.tsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)

```

3.2 Context

File: client/src/context/AuthContext.tsx

```

import React, { createContext, useState, useEffect, useContext } from 'react';
import { useNavigate } from 'react-router-dom';

interface AuthContextType {
  user: any;
  login: (userData: any, token: string) => void;
  logout: () => void;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

```

```

export const AuthProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const [user, setUser] = useState<any>(null);
  const navigate = useNavigate();

  useEffect(() => {
    const profile = localStorage.getItem('profile');
    if (profile) {
      setUser(JSON.parse(profile).result);
    }
  }, []);

  const login = (userData: any, token: string) => {
    localStorage.setItem('profile', JSON.stringify({ result: userData, token }));
    setUser(userData);
    navigate('/dashboard');
  };

  const logout = () => {
    localStorage.clear();
    setUser(null);
    navigate('/');
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be used within an AuthProvider');
  return context;
};

```

3.3 Pages

File: `client/src/pages/CourseDetail.tsx`

```

import React, { useEffect, useState } from 'react';
import { useParams } from 'react-router-dom';
import { fetchCourse, fetchLessons } from '../api';
import API from '../api';
import TakeQuiz from '../components/TakeQuiz';
import { PlayCircle } from 'lucide-react';

const CourseDetail: React.FC = () => {
  const { id } = useParams<{ id: string }>();
  const [course, setCourse] = useState<any>(null);

```

```

const [lessons, setLessons] = useState<any[]>([]);
const [activeLesson, setActiveLesson] = useState<any>(null);

const [quizzes, setQuizzes] = useState<any[]>([]);
const [showQuiz, setShowQuiz] = useState<any>(null);

useEffect(() => {
  if (id) {
    loadData();
  }
}, [id]);

const loadData = async () => {
  try {
    const courseRes = await fetchCourse(id!);
    setCourse(courseRes.data);
    const lessonsRes = await fetchLessons(id!);
    setLessons(lessonsRes.data);
    if (lessonsRes.data.length > 0) setActiveLesson(lessonsRes.data[0]);

    // Fetch quizzes
    const quizzesRes = await API.get(`/quizzes/course/${id}`);
    setQuizzes(quizzesRes.data);
  } catch (error) {
    console.error(error);
  }
}

if (!course) return <div className="p-10 text-center">Loading course...</div>;

if (showQuiz) {
  return (
    <div className="fixed inset-0 bg-black bg-opacity-50 flex items-center justify-center p-4 z-50">
      <TakeQuiz quiz={showQuiz} onClose={() => setShowQuiz(null)} />
    </div>
  )
}

return (
  <div className="max-w-7xl mx-auto py-10 px-4">
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
      {/* Main Content Area */}
      <div className="lg:col-span-2 space-y-8">
        {/* Video Player */}
        <div className="bg-black rounded-xl overflow-hidden shadow-lg aspect-video flex items-center justify-center">
          {activeLesson ? (
            <iframe
              src={activeLesson.videoUrl.replace('watch?v=', 'embed/')}
              title={activeLesson.title}
              className="w-full h-full"
            />
          ) : null}
        </div>
      </div>
    </div>
  </div>
)

```

```

        frameBorder="0"
        allowFullScreen
    ></iframe>
  ) : (
    <div className="text-white">Select a lesson to start
watching</div>

  )}
</div>

  { /* Lesson Info */}
  <div className="bg-white p-6 rounded-lg shadow-sm">
    <h2 className="text-2xl font-bold text-gray-900">
{activeLesson?.title || course.title}</h2>
    <p className="text-gray-600 mt-2">{activeLesson?.notes ||
course.description}</p>
  </div>
</div>

  { /* Sidebar / Lesson List */}
  <div className="lg:col-span-1">
    <div className="bg-white rounded-lg shadow-lg overflow-hidden sticky
top-24">
      <div className="p-4 bg-gray-50 border-b border-gray-200">
        <h3 className="font-bold text-lg text-gray-800">Course
Content</h3>
        <p className="text-sm text-gray-500">{lessons.length}
Lessons</p>
      </div>
      <div className="max-h-[600px] overflow-y-auto">
        {lessons.map((lesson, index) => (
          <div
            key={lesson._id}
            onClick={() => setActiveLesson(lesson)}
            className={`p-4 border-b border-gray-100 cursor-pointer
hover:bg-gray-50 transition ${activeLesson?._id === lesson._id ? 'bg-indigo-50 border-l-4
border-l-primary' : ''}`}
          >
            <div className="flex items-start">
              <span className="mt-1 mr-3 text-gray-400 text-sm
font-mono">{index + 1}</span>
              <div>
                <h4 className={`text-sm font-medium
${activeLesson?._id === lesson._id ? 'text-primary' : 'text-gray-700'}`}>{lesson.title}</h4>
                <div className="flex items-center mt-1 text-xs
text-gray-400">
                  <PlayCircle className="w-3 h-3 mr-1" />
                  <span>Video</span>
                </div>
              </div>
            </div>
          </div>
        )
        </div>
      )}
    </div>
  )}

```

```

    </div>

    { /* Quizzes List */ }
    <div className="p-4 bg-gray-50 border-t border-b border-gray-200 mt-
4">

        <h3 className="font-bold text-lg text-gray-800">Quizzes</h3>
    </div>
    <div className="p-4 space-y-3">
        { quizzes.length === 0 ? <p className="text-gray-500 text-sm">No
quizzes available.</p> : (
            quizzes.map((quiz) => (
                <button
                    key={quiz._id}
                    onClick={() => setShowQuiz(quiz)}
                    className="w-full text-left p-3 border rounded-md
hover:bg-gray-50 text-sm font-medium text-primary flex justify-between items-center"
                >
                    <span>{quiz.title}</span>
                    <span className="bg-indigo-100 text-indigo-800 text-
xs px-2 py-1 rounded-full">Start</span>
                </button>
            ))
        ) }
    </div>
</div>
</div>
</div>
</div>
    );
};

export default CourseDetail;

```

File: client/src/pages/CourseList.tsx

```
import React, { useEffect, useState } from 'react';
import { fetchCourses } from '../api';
import { Link } from 'react-router-dom';

const CourseList: React.FC = () => {
  const [courses, setCourses] = useState<any[]>([]);

  useEffect(() => {
    const loadCourses = async () => {
      try {
        const { data } = await fetchCourses();
        setCourses(data);
      } catch (error) {
        console.error(error);
      }
    };
  });
}
```

```

        loadCourses();
    }, []);

    return (
        <div className="max-w-7xl mx-auto py-10 px-4">
            <h1 className="text-3xl font-bold text-gray-900 mb-8">Browse Courses</h1>
            <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-8">
                {courses.map((course) => (
                    <div key={course._id} className="bg-white rounded-lg shadow-md overflow-hidden hover:shadow-xl transition-shadow duration-300">
                        <img src={course.thumbnail || 'https://via.placeholder.com/300'} alt={course.title} className="w-full h-48 object-cover" />
                        <div className="p-6">
                            <span className="text-xs font-semibold text-primary uppercase tracking-wider">{course.category}</span>
                            <h3 className="text-xl font-bold text-gray-900 mt-2 mb-2">{course.title}</h3>
                            <p className="text-gray-600 text-sm mb-4 line-clamp-2">{course.description}</p>
                            <div className="flex justify-between items-center">
                                <span className="text-sm text-gray-500">By {course.instructor?.name}</span>
                                <Link to={` /courses/${course._id}`} className="bg-primary text-white px-4 py-2 rounded-md hover:bg-indigo-700 text-sm font-medium">View Course</Link>
                            </div>
                        </div>
                    </div>
                ))}
            </div>
        </div>
    );
};

export default CourseList;

```

File: client/src/pages/CreateCourse.tsx

```

import React, { useState } from 'react';
import { createCourse } from '../api';
import { useNavigate } from 'react-router-dom';

const CreateCourse: React.FC = () => {
    const [formData, setFormData] = useState({ title: '', description: '', category: '', thumbnail: '' });
    const navigate = useNavigate();
    const [error, setError] = useState('');

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
    }

```

```

    try {
      const { data } = await createCourse(formData);
      navigate(`/manage-course/${data._id}`);
    } catch (err: any) {
      setError(err.response?.data?.message || 'Error creating course');
    }
  };

  return (
    <div className="max-w-2xl mx-auto py-10 px-4">
      <h1 className="text-3xl font-bold mb-8">Create New Course</h1>
      {error && <div className="text-red-500 mb-4">{error}</div>}
      <form onSubmit={handleSubmit} className="space-y-6">
        <div>
          <label className="block text-sm font-medium text-gray-700">Course
Title</label>
          <input
            type="text"
            required
            className="mt-1 block w-full border border-gray-300 rounded-md
shadow-sm py-2 px-3 focus:outline-none focus:ring-primary focus:border-primary"
            value={formData.title}
            onChange={(e) => setFormData({ ...formData, title: e.target.value
}})
          />
        </div>
        <div>
          <label className="block text-sm font-medium text-gray-
700">Description</label>
          <textarea
            required
            rows={4}
            className="mt-1 block w-full border border-gray-300 rounded-md
shadow-sm py-2 px-3 focus:outline-none focus:ring-primary focus:border-primary"
            value={formData.description}
            onChange={(e) => setFormData({ ...formData, description:
e.target.value }}}
          />
        </div>
        <div>
          <label className="block text-sm font-medium text-gray-
700">Category</label>
          <input
            type="text"
            required
            className="mt-1 block w-full border border-gray-300 rounded-md
shadow-sm py-2 px-3 focus:outline-none focus:ring-primary focus:border-primary"
            value={formData.category}
            onChange={(e) => setFormData({ ...formData, category: e.target.value
}})
          />
        </div>
      </form>
    </div>
  );

```

```

        <div>
          <label className="block text-sm font-medium text-gray-700">Thumbnail
URL</label>
          <input
            type="text"
            className="mt-1 block w-full border border-gray-300 rounded-md
shadow-sm py-2 px-3 focus:outline-none focus:ring-primary focus:border-primary"
            value={formData.thumbnail}
            onChange={(e) => setFormData({ ...formData, thumbnail:
e.target.value })}
            placeholder="https://example.com/image.jpg"
          />
        </div>
        <button
          type="submit"
          className="w-full flex justify-center py-2 px-4 border border-
transparent rounded-md shadow-sm text-sm font-medium text-white bg-primary hover:bg-indigo-
700 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-primary"
        >
          Create & Continue
        </button>
      </form>
    </div>
  );
};

export default CreateCourse;

```

File: client/src/pages/CreateQuiz.tsx

```

import React, { useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import API from '../api';

const CreateQuiz: React.FC = () => {
  const { courseId } = useParams<{ courseId: string }>();
  const navigate = useNavigate();
  const [title, setTitle] = useState('');
  const [questions, setQuestions] = useState([
    { questionText: '', options: ['', '', '', ''],
correctAnswer: '' }
  ]);

  // Static Selection State
  const [selectedCourse, setSelectedCourse] = useState('');
  const [selectedBranch, setSelectedBranch] = useState('');

  const handleQuestionChange = (index: number, field: string, value: string) => {
    const newQuestions = [...questions];
    if (field === 'questionText') newQuestions[index].questionText = value;
    else if (field === 'correctAnswer') newQuestions[index].correctAnswer = value;
    setQuestions(newQuestions);
  };
};

```

```

const handleOptionChange = (qIndex: number, oIndex: number, value: string) => {
  const newQuestions = [...questions];
  newQuestions[qIndex].options[oIndex] = value;
  setQuestions(newQuestions);
};

const addQuestion = () => {
  setQuestions([...questions, { questionText: '', options: ['', '', '', ''],
correctAnswer: '' }]);
};

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();

  // Validation for general quiz
  if (courseId === 'general') {
    if (!selectedCourse) {
      alert('Please select a Target Course (e.g., B.Tech).');
      return;
    }
    if ((selectedCourse === 'B.Tech' || selectedCourse === 'M.Tech') &&
!selectedBranch) {
      alert('Please select a Target Branch.');
```

```

return (
  <div className="max-w-3xl mx-auto py-10 px-4">
    <h1 className="text-3xl font-bold mb-6">Create Quiz</h1>
    <form onSubmit={handleSubmit} className="space-y-6">

      {courseId === 'general' && (
        <div className="bg-white p-6 rounded-lg shadow border border-gray-200
space-y-4">

          <h3 className="font-semibold text-lg">Target Audience</h3>

          <div>
            <label className="block text-sm font-medium text-gray-
700">Target Course</label>
            <select
md p-2"
              value={selectedCourse}
              onChange={(e) => setSelectedCourse(e.target.value)}
              required
            >
              <option value="">Select Course</option>
              <option value="B.Tech">B.Tech</option>
              <option value="M.Tech">M.Tech</option>
              <option value="BCA">BCA</option>
              <option value="MCA">MCA</option>
            </select>
          </div>

          {(selectedCourse === 'B.Tech' || selectedCourse === 'M.Tech') && (
            <div>
              <label className="block text-sm font-medium text-gray-
700">Target Branch</label>
              <select
rounded-md p-2"
                value={selectedBranch}
                onChange={(e) => setSelectedBranch(e.target.value)}
                required
              >
                <option value="">Select Branch</option>
                <option value="CSE">CSE</option>
                <option value="CSD">CSD</option>
                <option value="AIML">AIML</option>
                <option value="Mechanical">Mechanical</option>
                <option value="Civil">Civil</option>
              </select>
            </div>
          )}
        </div>
      )}
    </form>
  </div>
)

```

```

<label className="block text-sm font-medium text-gray-700">Quiz
Title</label>

<input
  type="text"
  required
  className="mt-1 block w-full border border-gray-300 rounded-md p-2"
  value={title}
  onChange={(e) => setTitle(e.target.value)}
/>
</div>

{questions.map((q, qIndex) => (
  <div key={qIndex} className="bg-white p-6 rounded-lg shadow border
border-gray-200">
    <h4 className="font-bold mb-2">Question {qIndex + 1}</h4>
    <input
      type="text"
      placeholder="Question Text"
      required
      className="block w-full border border-gray-300 rounded-md p-2
mb-4"
      value={q.questionText}
      onChange={(e) => handleQuestionChange(qIndex, 'questionText',
e.target.value)}
    />
    <div className="grid grid-cols-2 gap-4 mb-4">
      {q.options.map((opt, oIndex) => (
        <input
          key={oIndex}
          type="text"
          placeholder={`Option ${oIndex + 1}`}
          required
          className="block w-full border border-gray-300 rounded-
md p-2"
          value={opt}
          onChange={(e) => handleOptionChange(qIndex, oIndex,
e.target.value)}
        />
      ))}
    </div>
    <input
      type="text"
      placeholder="Correct Answer (must match one option exactly)"
      required
      className="block w-full border border-gray-300 rounded-md p-2
bg-green-50"
      value={q.correctAnswer}
      onChange={(e) => handleQuestionChange(qIndex, 'correctAnswer',
e.target.value)}
    />
  </div>
))}

```

```

        <button type="button" onClick={addQuestion} className="text-primary font-
medium hover:underline">
            + Add Question
        </button>

        <button type="submit" className="w-full bg-primary text-white py-3 rounded-
md font-bold hover:bg-indigo-700">
            Save Quiz
        </button>
    </form>
</div>
);
};

export default CreateQuiz;

```

File: client/src/pages/Dashboard.tsx

```

import React from 'react';
import { useAuth } from '../context/AuthContext';
import AdminDashboard from '../components/AdminDashboard';
import StudentDashboard from '../components/StudentDashboard';
import TeacherDashboard from '../components/TeacherDashboard';

const Dashboard: React.FC = () => {
    const { user } = useAuth();

    if (!user) return <div className="text-center mt-20">Please log in to view dashboard.
</div>;

    return (
        <div className="min-h-screen bg-gray-50 py-10">
            <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
                <h1 className="text-3xl font-bold text-gray-900 mb-8">Welcome back,
{user.name}</h1>
                {user.role === 'admin' ? <AdminDashboard /> : user.role === 'teacher' ?
<TeacherDashboard /> : <StudentDashboard />}
            </div>
        </div>
    );
};

export default Dashboard;

```

File: client/src/pages/Home.tsx

```

import React from 'react';
import { Link } from 'react-router-dom';

```

```

import Footer from '../components/Footer';
import { BASE_URL } from '../api';

const Home: React.FC = () => {
  // Add a timestamp to bust cache
  const [brandName, setBrandName] = React.useState('EduPortal');
  const [bgUrl, setBgUrl] = React.useState('');

  React.useEffect(() => {
    const fetchSettings = async () => {
      try {
        // We can import axios or just use fetch for zero-dependency if we didn't
import axios in file
        // But let's assume axios is available or use fetch
        const res = await fetch(`${BASE_URL}/api/site-settings`);
        const data = await res.json();
        if (data) {
          if (data.brandName) setBrandName(data.brandName);
          if (data.backgroundUrl) {
            setBgUrl(data.backgroundUrl.startsWith('http') ? data.backgroundUrl
: `${BASE_URL}${data.backgroundUrl}`);
          }
        }
      } catch (error) {
        console.error('Failed to fetch site settings', error);
      }
    };
    fetchSettings();
  }, []);

  return (
    <div className="flex flex-col min-h-screen">
      <style>
        {`
          @keyframes kenburns {
            0% { transform: scale(1) translate(0, 0); }
            50% { transform: scale(1.1) translate(-1%, -1%); }
            100% { transform: scale(1) translate(0, 0); }
          }
          .animate-kenburns {
            animation: kenburns 20s ease-in-out infinite alternate;
          }
        `}
      </style>
      <div className="flex-grow relative overflow-hidden flex flex-col justify-center
items-center">
        {/* Animated Background */}
        <div
          className="absolute inset-0 bg-cover bg-center bg-no-repeat animate-
kenburns z-0"
          style={{
            backgroundImage: `url(${bgUrl})`,

```

```

        backgroundColor: '#f9fafb'
      }}
    />

    {/* Overlay to ensure text readability */}
    <div className="absolute inset-0 bg-white/80 z-0"></div>

    <div className="text-center max-w-2xl px-4 z-10 relative">
      <h1 className="text-5xl font-extrabold text-gray-900 mb-6 tracking-
tight">
        Master New Skills with <span className="text-primary">{brandName}
</span>

      </h1>
      <p className="text-xl text-gray-800 mb-8 font-medium">
        The ultimate platform for students and teachers. Learn at your own
pace, track your progress, and achieve your goals.
      </p>
      <div className="flex justify-center gap-4">
        <Link to="/register" className="bg-primary text-white px-8 py-3
rounded-full font-semibold text-lg hover:bg-indigo-700 transition shadow-lg">
          Get Started
        </Link>
      </div>
    </div>
    <div>
      <Footer />
    </div>
  );
};

export default Home;

```

File: client/src/pages/Login.tsx

```

import React, { useState } from 'react';
import { useAuth } from '../context/AuthContext';
import { signIn } from '../api';
import { useNavigate } from 'react-router-dom';

const Login: React.FC = () => {
  const [formData, setFormData] = useState({ email: '', password: '', role: 'student' });
  const { login } = useAuth();
  const navigate = useNavigate();
  const [error, setError] = useState('');

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      const { data } = await signIn(formData);
      login({ ...data.result, role: formData.role }, data.token);
      navigate('/dashboard');
    }
  }
};

```

```

    } catch (err: any) {
      setError(err.response?.data?.message || 'Something went wrong');
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-50 py-12 px-4 sm:px-6 lg:px-8">
      <div className="max-w-md w-full space-y-8 bg-white p-8 rounded-xl shadow-lg">
        <div>
          <h2 className="mt-6 text-center text-3xl font-extrabold text-gray-900">Sign in to your account</h2>
        </div>
        <form className="mt-8 space-y-6" onSubmit={handleSubmit}>
          {error && <div className="text-red-500 text-center text-sm">{error}</div>}

          <div className="rounded-md shadow-sm -space-y-px">
            <div>
              <input
                type="email"
                required
                className="appearance-none rounded-none relative block w-full px-3 py-2 border border-gray-300 placeholder-gray-500 text-gray-900 rounded-t-md focus:outline-none focus:ring-primary focus:border-primary focus:z-10 sm:text-sm"
                placeholder="Email address"
                onChange={(e) => setFormData({ ...formData, email: e.target.value }}}
              />
            </div>
            <div>
              <input
                type="password"
                required
                className="appearance-none rounded-none relative block w-full px-3 py-2 border border-gray-300 placeholder-gray-500 text-gray-900 focus:outline-none focus:ring-primary focus:border-primary focus:z-10 sm:text-sm"
                placeholder="Password"
                onChange={(e) => setFormData({ ...formData, password: e.target.value }}}
              />
            </div>
            <div>
              <select
                className="appearance-none rounded-none relative block w-full px-3 py-2 border border-gray-300 placeholder-gray-500 text-gray-900 rounded-b-md focus:outline-none focus:ring-primary focus:border-primary focus:z-10 sm:text-sm"
                value={formData.role}
                onChange={(e) => setFormData({ ...formData, role: e.target.value }}}
              >
                <option value="student">Student</option>
                <option value="teacher">Teacher</option>
              </select>
            </div>
          </div>
        </form>
      </div>
    </div>
  );
}

```

```

        <option value="admin">Admin</option>
      </select>
    </div>
  </div>

  <div>
    <button
      type="submit"
      className="group relative w-full flex justify-center py-2 px-4
border border-transparent text-sm font-medium rounded-md text-white bg-primary hover:bg-
indigo-700 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-primary"
    >
      Sign in
    </button>
  </div>
</form>
</div>
</div>
);
};

export default Login;

```

File: client/src/pages/ManageCourse.tsx

```

import React, { useEffect, useState } from 'react';
import { useParams } from 'react-router-dom';
import { fetchCourse, fetchLessons, createLesson } from '../api';

const ManageCourse: React.FC = () => {
  const { id } = useParams<{ id: string }>();
  const [course, setCourse] = useState<any>(null);
  const [lessons, setLessons] = useState<any[]>([]);
  const [showAddLesson, setShowAddLesson] = useState(false);
  const [newLesson, setNewLesson] = useState({ title: '', videoUrl: '', notes: '' });

  useEffect(() => {
    if (id) {
      loadData();
    }
  }, [id]);

  const loadData = async () => {
    try {
      const courseRes = await fetchCourse(id!);
      setCourse(courseRes.data);
      const lessonsRes = await fetchLessons(id!);
      setLessons(lessonsRes.data);
    } catch (error) {
      console.error(error);
    }
  }
}

```

```

}

const handleAddLesson = async (e: React.FormEvent) => {
  e.preventDefault();
  try {
    await createLesson({ ...newLesson, courseId: id });
    setNewLesson({ title: '', videoUrl: '', notes: '' });
    setShowAddLesson(false);
    loadData();
  } catch (error) {
    console.error(error);
  }
};

if (!course) return <div>Loading...</div>;

return (
  <div className="max-w-5xl mx-auto py-10 px-4">
    <div className="flex justify-between items-start mb-8">
      <div>
        <h1 className="text-3xl font-bold text-gray-900">{course.title}</h1>
        <p className="text-gray-600 mt-2">{course.description}</p>
      </div>
      <div className="flex space-x-4">
        <button onClick={() => setShowAddLesson(!showAddLesson)} className="bg-primary text-white px-4 py-2 rounded-md hover:bg-indigo-700">
          {showAddLesson ? 'Cancel' : 'Add Lesson'}
        </button>
        <a href={` /create-quiz/${id}`} className="bg-indigo-500 text-white px-4 py-2 rounded-md hover:bg-indigo-600 block">
          Create Quiz
        </a>
      </div>
    </div>

    {showAddLesson && (
      <div className="bg-white p-6 rounded-lg shadow mb-8 border border-gray-200">
        <h3 className="text-lg font-medium mb-4">Add New Lesson</h3>
        <form onSubmit={handleAddLesson} className="space-y-4">
          <input
            type="text"
            placeholder="Lesson Title"
            required
            className="block w-full border border-gray-300 rounded-md p-2"
            value={newLesson.title}
            onChange={(e) => setNewLesson({ ...newLesson, title:
e.target.value }}}
          />
          <input
            type="text"
            placeholder="Video URL (YouTube/Vimeo)"
            required

```

```

        className="block w-full border border-gray-300 rounded-md p-2"
        value={newLesson.videoUrl}
        onChange={(e) => setNewLesson({ ...newLesson, videoUrl:
e.target.value })}

        />
        <textarea
        placeholder="Notes"
        className="block w-full border border-gray-300 rounded-md p-2"
        value={newLesson.notes}
        onChange={(e) => setNewLesson({ ...newLesson, notes:
e.target.value })}

        />
        <button type="submit" className="bg-green-600 text-white px-4 py-2
rounded-md hover:bg-green-700">Save Lesson</button>
    </form>
  </div>
)}

<div className="space-y-4">
  <h2 className="text-2xl font-bold text-gray-800">Lessons ({lessons.length})
</h2>

  {lessons.length === 0 ? <p>No lessons added yet.</p> : (
    lessons.map((lesson, index) => (
      <div key={lesson._id} className="bg-white p-4 rounded-lg shadow-sm
border border-gray-100 flex justify-between items-center">
        <div className="flex items-center">
          <span className="bg-gray-100 text-gray-600 w-8 h-8 flex
items-center justify-center rounded-full mr-4 font-bold">{index + 1}</span>
          <div>
            <h4 className="font-semibold text-lg">{lesson.title}
</h4>

            <a href={lesson.videoUrl} target="_blank" rel="noopener
noreferrer" className="text-indigo-600 text-sm hover:underline">Watch Video</a>
          </div>
        </div>
      </div>
    ))
  )}
</div>
</div>
);
};

export default ManageCourse;

```

File: client/src/pages/Register.tsx

```

import React, { useState } from 'react';
import { signUp } from '../api';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

```

```

const Register: React.FC = () => {
  const [formData, setFormData] = useState({ name: '', email: '', password: '', role:
'student', course: '', branch: '', fatherName: '', motherName: '' });
  const [error, setError] = useState('');
  const navigate = useNavigate();
  const { login } = useAuth();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      const { data } = await signUp(formData);

      // If explicit message about approval, or no token, show message and redirect to
login
      if (!data.token) {
        alert(data.message || "Registration successful. Please wait for admin
approval.");
        navigate('/login');
        return;
      }

      // Auto login after register ONLY if token exists (Admin only basically)
      login({ ...data.result, role: formData.role }, data.token);
      navigate('/dashboard');
    } catch (err: any) {
      setError(err.response?.data?.message || 'Something went wrong');
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-50 py-12 px-4
sm:px-6 lg:px-8">
      <div className="max-w-md w-full space-y-8 bg-white p-8 rounded-xl shadow-lg">
        <div>
          <h2 className="mt-6 text-center text-3xl font-extrabold text-gray-
900">Create your account</h2>
        </div>
        <form className="mt-8 space-y-6" onSubmit={handleSubmit}>
          {error && <div className="text-red-500 text-center text-sm">{error}</div>}
        </div>

        <div className="rounded-md shadow-sm -space-y-px">
          <div>
            <input
              type="text"
              required
              className="appearance-none rounded-none relative block w-
full px-3 py-2 border border-gray-300 placeholder-gray-500 text-gray-900 rounded-t-md
focus:outline-none focus:ring-primary focus:border-primary focus:z-10 sm:text-sm"
              placeholder="Full Name"
              onChange={(e) => setFormData({ ...formData, name:
e.target.value }}})

```

```

        />
      </div>
      <div>
        <input
          type="email"
          required
          className="appearance-none rounded-none relative block w-
full px-3 py-2 border border-gray-300 placeholder-gray-500 text-gray-900 focus:outline-none
focus:ring-primary focus:border-primary focus:z-10 sm:text-sm"
          placeholder="Email address"
          onChange={(e) => setFormData({ ...formData, email:
e.target.value }}}
        />
      </div>
      <div>
        <input
          type="password"
          required
          className="appearance-none rounded-none relative block w-
full px-3 py-2 border border-gray-300 placeholder-gray-500 text-gray-900 focus:outline-none
focus:ring-primary focus:border-primary focus:z-10 sm:text-sm"
          placeholder="Password"
          onChange={(e) => setFormData({ ...formData, password:
e.target.value }}}
        />
      </div>
      <div>
        <select
          className="appearance-none rounded-none relative block w-
full px-3 py-2 border border-gray-300 placeholder-gray-500 text-gray-900 focus:outline-none
focus:ring-primary focus:border-primary focus:z-10 sm:text-sm"
          value={formData.role}
          onChange={(e) => setFormData({ ...formData, role:
e.target.value }}}
        >
          <option value="student">Student</option>
          <option value="teacher">Teacher</option>
          <option value="admin">Admin</option>
        </select>
      </div>
      <div>
        <select
          className="appearance-none rounded-none relative block w-
full px-3 py-2 border border-gray-300 placeholder-gray-500 text-gray-900 focus:outline-none
focus:ring-primary focus:border-primary focus:z-10 sm:text-sm"
          value={formData.course}
          onChange={(e) => setFormData({ ...formData, course:
e.target.value }}}
          required
        >
          <option value="">Select Course</option>
          <option value="B.Tech">B.Tech</option>

```

```

        <option value="M.Tech">M.Tech</option>
        <option value="BCA">BCA</option>
        <option value="MCA">MCA</option>
      </select>
    </div>
    {(formData.course === 'B.Tech' || formData.course === 'M.Tech') && (
      <div>
        <select
          className="appearance-none rounded-none relative block
w-full px-3 py-2 border border-gray-300 placeholder-gray-500 text-gray-900 focus:outline-
none focus:ring-primary focus:border-primary focus:z-10 sm:text-sm"
          value={formData.branch}
          onChange={(e) => setFormData({ ...formData, branch:
e.target.value }}}
          required
        >
          <option value="">Select Branch</option>
          <option value="CSE">CSE</option>
          <option value="CSD">CSD</option>
          <option value="AIML">AIML</option>
          <option value="Mechanical">Mechanical</option>
          <option value="Civil">Civil</option>
        </select>
      </div>
    )}
    {(formData.role === 'student' && (
      <>
        <div>
          <input
            type="text"
            required
            className="appearance-none rounded-none relative
block w-full px-3 py-2 border border-gray-300 placeholder-gray-500 text-gray-900
focus:outline-none focus:ring-primary focus:border-primary focus:z-10 sm:text-sm"
            placeholder="Father's Name"
            onChange={(e) => setFormData({ ...formData,
fatherName: e.target.value }}}
          />
        </div>
        <div>
          <input
            type="text"
            required
            className="appearance-none rounded-none relative
block w-full px-3 py-2 border border-gray-300 placeholder-gray-500 text-gray-900 rounded-b-
md focus:outline-none focus:ring-primary focus:border-primary focus:z-10 sm:text-sm"
            placeholder="Mother's Name"
            onChange={(e) => setFormData({ ...formData,
motherName: e.target.value }}}
          />
        </div>
      </>
    )}
  </div>
</>

```

```

        })
      </div>

      <div>
        <button
          type="submit"
          className="group relative w-full flex justify-center py-2 px-4
border border-transparent text-sm font-medium rounded-md text-white bg-primary hover:bg-
indigo-700 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-primary"
        >
          Sign up
        </button>
      </div>
    </form>
  </div>
</div>
);
};

export default Register;

```

File: client/src/pages/TakeQuiz.tsx

```

import React, { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import API from '../api';

const TakeQuiz: React.FC = () => {
  const { id } = useParams<{ id: string }>();
  const navigate = useNavigate();
  const [quiz, setQuiz] = useState<any>(null);
  const [loading, setLoading] = useState(true);
  const [answers, setAnswers] = useState<{ [key: number]: string }>({});
  const [result, setResult] = useState<{ score: number, total: number } | null>(null);

  useEffect(() => {
    const fetchQuiz = async () => {
      try {
        const { data } = await API.get(`/quizzes/${id}`);
        setQuiz(data);
        setLoading(false);
      } catch (error) {
        console.error('Error fetching quiz:', error);
        alert('Failed to load quiz');
        setLoading(false);
      }
    };
    fetchQuiz();
  }, [id]);

  const handleOptionSelect = (qIndex: number, option: string) => {

```

```

    setAnswers({ ...answers, [qIndex]: option });
  });

const handleSubmit = async () => {
  // Validation: Ensure all questions are answered
  if (quiz && Object.keys(answers).length !== quiz.questions.length) {
    alert('Please answer all questions before submitting.');
```

return;

}

```

  try {
    const { data } = await API.post('/quizzes/submit', {
      quizId: id,
      answers // sending object { 0: 'Option A', 1: 'Option B' }
    });
    setResult(data);
  } catch (error: any) {
    console.error('Error submitting quiz:', error);
    alert('Failed to submit quiz');
  }
};

if (loading) return <div className="p-8 text-center">Loading quiz...</div>;
if (!quiz) return <div className="p-8 text-center">Quiz not found</div>;

if (result) {
  return (
    <div className="max-w-2xl mx-auto py-10 px-4 text-center">
      <div className="bg-white p-8 rounded-lg shadow-lg border-t-4 border-indigo-500">
        <h2 className="text-3xl font-bold text-gray-800 mb-4">Quiz Completed!
      </h2>

      <p className="text-xl text-gray-600 mb-6">You scored:</p>
      <div className="text-6xl font-bold text-indigo-600 mb-6">
        {result.score} / {result.total}
      </div>
      <div className="flex justify-center gap-4">
        <button
          onClick={() => navigate('/dashboard')}
          className="bg-gray-500 text-white px-6 py-2 rounded-md hover:bg-gray-600"
        >
          Back to Dashboard
        </button>
      </div>
    </div>
  );
}

return (
  <div className="max-w-3xl mx-auto py-10 px-4">

```

```

<div className="mb-6">
  <h1 className="text-3xl font-bold text-gray-900">{quiz.title}</h1>
  <p className="text-gray-500 mt-1">
    {quiz.questions.length} Questions • {quiz.course} {quiz.branch ? `•
    ${quiz.branch}` : ''}
  </p>
</div>

<div className="space-y-6">
  {quiz.questions.map((q: any, index: number) => (
    <div key={index} className="bg-white p-6 rounded-lg shadow border
border-gray-200">
      <h3 className="font-semibold text-lg mb-4">
        {index + 1}. {q.questionText}
      </h3>
      <div className="space-y-2">
        {q.options.map((option: string, oIndex: number) => (
          <label
            key={oIndex}
            className={`flex items-center p-3 rounded-md border
cursor-pointer transition ${answers[index] === option
? 'bg-indigo-50 border-indigo-500 ring-1 ring-
indigo-500'
: 'bg-gray-50 border-gray-200 hover:bg-gray-100'
} `}
          >
            <input
              type="radio"
              name={`question-${index}`}
              value={option}
              checked={answers[index] === option}
              onChange={() => handleOptionSelect(index, option)}
              className="h-4 w-4 text-indigo-600 focus:ring-
indigo-500 border-gray-300"
            />
            <span className="ml-3 text-gray-700">{option}</span>
          </label>
        ))}
      </div>
    </div>
  ))}
</div>

<div className="mt-8 flex justify-end">
  <button
    onClick={handleSubmit}
    className="bg-indigo-600 text-white px-8 py-3 rounded-md font-bold text-
lg hover:bg-indigo-700 shadow-md transition transform hover:-translate-y-0.5"
  >
    Submit Quiz
  </button>
</div>

```

```

        </div>
    );
};

export default TakeQuiz;

```

3.4 Components

File: `client/src/components/AdminDashboard.tsx`

```

import React, { useEffect, useState } from 'react';
import API, { BASE_URL } from '../api';

const AdminDashboard: React.FC = () => {
    const [stats, setStats] = useState({ studentCount: 0, teacherCount: 0, courseCount: 0 });

    const [pendingTeachers, setPendingTeachers] = useState<any[]>([]);
    const [pendingStudents, setPendingStudents] = useState<any[]>([]);
    const [siteSettings, setSiteSettings] = useState({
        brandName: '',
        logoUrl: '',
        backgroundUrl: '',
        githubLink: '',
        linkedinLink: '',
        copyrightText: ''
    });

    useEffect(() => {
        loadData();
    }, []);

    const loadData = async () => {
        try {
            const statsRes = await API.get('/admin/stats');
            setStats(statsRes.data);
            const teachersRes = await API.get('/admin/teachers/pending');
            setPendingTeachers(teachersRes.data);
            const studentsRes = await API.get('/admin/students/pending');
            setPendingStudents(studentsRes.data);
            const settingsRes = await API.get('/site-settings');
            setSiteSettings(settingsRes.data);
        } catch (error) {
            console.error(error);
        }
    }

    const handleApprove = async (id: string, role: 'teacher' | 'student') => {
        try {
            await API.put(`/admin/${role}s/${id}/approve`);
            loadData();
            alert(`${role.charAt(0).toUpperCase() + role.slice(1)} Approved successfully! An

```

```

email notification has been sent.`);
    } catch (error: any) {
      console.error(error);
      const errorMessage = error.response?.data?.message || 'Error approving user';
      alert(`Failed to approve user: ${errorMessage}`);
    }
  }

  const handleReject = async (id: string, role: 'teacher' | 'student') => {
    if (!window.confirm("Are you sure you want to reject this user?")) return;
    try {
      await API.put(`/admin/${role}s/${id}/reject`);
      loadData();
      alert(`${role.charAt(0).toUpperCase() + role.slice(1)} Rejected successfully! An
email notification has been sent.`);
    } catch (error: any) {
      console.error(error);
      const errorMessage = error.response?.data?.message || 'Error rejecting user';
      alert(`Failed to reject user: ${errorMessage}`);
    }
  }

  const renderTable = (users: any[], role: 'teacher' | 'student', title: string) => (
    <div className="mb-10">
      <h3 className="text-xl font-bold mb-4">{title}</h3>
      {users.length === 0 ? (
        <div className="bg-white p-6 rounded-lg shadow text-center text-gray-500">No
pending approvals</div>
      ) : (
        <div className="bg-white shadow rounded-lg overflow-hidden">
          <table className="min-w-full divide-y divide-gray-200">
            <thead className="bg-gray-50">
              <tr>
                <th className="px-6 py-3 text-left text-xs font-medium text-
gray-500 uppercase tracking-wider">Name</th>
                <th className="px-6 py-3 text-left text-xs font-medium text-
gray-500 uppercase tracking-wider">Email</th>
                <th className="px-6 py-3 text-right text-xs font-medium
text-gray-500 uppercase tracking-wider">Actions</th>
              </tr>
            </thead>
            <tbody className="bg-white divide-y divide-gray-200">
              {users.map((user) => (
                <tr key={user._id}>
                  <td className="px-6 py-4 whitespace-nowrap">{user.name}
</td>
                  <td className="px-6 py-4 whitespace-nowrap">{user.email}
</td>
                  <td className="px-6 py-4 whitespace-nowrap text-right
space-x-2">
                    <button
                      onClick={() => handleApprove(user._id, role)}

```

```

                                className="bg-green-600 text-white px-3 py-1
rounded text-sm hover:bg-green-700 transition"
                                >
                                    Approve
                                </button>
                                <button
                                    onClick={() => handleReject(user._id, role)}
                                    className="bg-red-600 text-white px-3 py-1
rounded text-sm hover:bg-red-700 transition"
                                >
                                    Reject
                                </button>
                            </td>
                        </tr>
                    </tbody>
                </table>
            </div>
        )}
    </div>
);

```

```

const handleBackgroundUpload = async (e: React.ChangeEvent<HTMLInputElement>) => {
    if (!e.target.files || e.target.files.length === 0) return;

    const file = e.target.files[0];
    const formData = new FormData();
    formData.append('image', file);

    try {
        const res = await API.post('/admin/upload-background', formData);

        // Update local state and auto-save the new background URL to site settings
        const newBgUrl = res.data.filePath;
        setSiteSettings(prev => ({ ...prev, backgroundUrl: newBgUrl }));
        await API.put('/site-settings', { ...siteSettings, backgroundUrl: newBgUrl });

        alert('Background image updated!');
        window.location.reload();
    } catch (error) {
        console.error('Error uploading background:', error);
        alert('Failed to update background image.');
    }
};

const handleDeleteBackground = async () => {
    if (!window.confirm("Are you sure you want to remove the background image?"))
return;
    try {
        await API.delete('/admin/background');
        setSiteSettings(prev => ({ ...prev, backgroundUrl: '' }));
        await API.put('/site-settings', { ...siteSettings, backgroundUrl: '' });
    }
};

```

```

        alert('Background image removed!');
        window.location.reload();
    } catch (error) {
        console.error('Error deleting background:', error);
        alert('Failed to delete background image.');
    }
};

const handleLogoUpload = async (e: React.ChangeEvent<HTMLInputElement>) => {
    if (!e.target.files || e.target.files.length === 0) return;

    const file = e.target.files[0];
    const formData = new FormData();
    formData.append('image', file);

    try {
        const res = await API.post('/admin/upload-logo', formData);

        // Update local state
        const newLogoUrl = res.data.filePath;
        setSiteSettings(prev => ({ ...prev, logoUrl: newLogoUrl }));

        // Auto-save the new logo URL to site settings
        await API.put('/site-settings', { ...siteSettings, logoUrl: newLogoUrl });

        alert('Logo updated and saved!');
        window.location.reload();
    } catch (error) {
        console.error('Error uploading logo:', error);
        alert('Failed to update logo.');
    }
};

const handleDeleteLogo = async () => {
    if (!window.confirm("Are you sure you want to remove the logo?")) return;
    try {
        await API.delete('/admin/logo');
        setSiteSettings(prev => ({ ...prev, logoUrl: '' }));
        alert('Logo removed!');
        window.location.reload();
    } catch (error) {
        console.error('Error deleting logo:', error);
        alert('Failed to delete logo.');
    }
};

const handleUpdateSettings = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
        await API.put('/site-settings', siteSettings);
        alert('Site settings updated successfully!');
    } catch (error) {

```

```

        console.error('Error updating site settings:', error);
        alert('Failed to update site settings.');
```

```
    }
```

```
};
```

```
return (
```

```
    <div>
```

```
        <div className="flex justify-between items-center mb-6">
```

```
            <h2 className="text-2xl font-bold">Admin Dashboard</h2>
```

```
            <button
```

```
                onClick={loadData}
```

```
                className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700
```

```
transition flex items-center"
```

```
            >
```

```
                 Refresh Data
```

```
            </button>
```

```
        </div>
```

```
        <div className="bg-white p-6 rounded-lg shadow mb-10">
```

```
            <h3 className="text-lg font-bold mb-4">Landing Page Settings</h3>
```

```
            <div className="mb-6">
```

```
                <h4 className="text-md font-semibold mb-2">Background Image</h4>
```

```
                <div className="mb-4 h-48 w-full bg-gray-200 rounded-lg overflow-hidden
```

```
relative border flex items-center justify-center">
```

```
                    {siteSettings.backgroundUrl ? (
```

```
                        <>
```

```
                            <img
```

```
                                src={siteSettings.backgroundUrl.startsWith('http') ?
```

```
siteSettings.backgroundUrl : `${BASE_URL}${siteSettings.backgroundUrl}`}
```

```
                                alt="Background Preview"
```

```
                                className="w-full h-full object-cover relative z-10"
```

```
                                onError={(e) => (e.currentTarget.style.display =
```

```
'none')}}

```

```
                            />
```

```
                            <div className="absolute bottom-0 left-0 bg-black/50 text-
```

```
white text-xs p-1 z-20">Current Background</div>
```

```
                        </>
```

```
                    ) : (
```

```
                        <span className="text-gray-500 absolute z-0">No Background
```

```
Image</span>
```

```
                    )}
```

```
                </div>
```

```
            <div className="flex items-center space-x-4">
```

```
                <label className="block text-sm font-medium text-gray-700">
```

```
                    Upload Background Image
```

```
                </label>
```

```
                <input
```

```
                    type="file"
```

```
                    accept="image/*"
```

```
                    onChange={handleBackgroundUpload}
```

```

        className="block w-full text-sm text-gray-500
        file:mr-4 file:py-2 file:px-4
        file:rounded-full file:border-0
        file:text-sm file:font-semibold
        file:bg-primary file:text-white
        hover:file:bg-indigo-700"
      />
      <button
        onClick={handleDeleteBackground}
        className="bg-red-600 text-white px-4 py-2 rounded text-sm
hover:bg-red-700 transition"
      >
        Remove
      </button>
    </div>
  </div>

  <div className="mb-6 border-t pt-6">
    <h4 className="text-md font-semibold mb-2">Brand Logo</h4>

    { /* Preview Logo */ }
    <div className="mb-4 flex items-center space-x-4">
      <div className="h-16 w-auto bg-gray-100 p-2 rounded border">
        {siteSettings.logoUrl ? (
          <img
            src={siteSettings.logoUrl.startsWith('http') ?
siteSettings.logoUrl : `${BASE_URL}${siteSettings.logoUrl}`}
            alt="Logo Preview"
            className="h-full w-auto"
            onError={(e) => (e.currentTarget.style.display =
'none'))}
          />
        ) : (
          <span className="text-gray-400 text-sm">No Logo</span>
        )}
      </div>
      <span className="text-sm text-gray-500">Current Logo Preview</span>
    </div>

    <div className="flex items-center space-x-4">
      <label className="block text-sm font-medium text-gray-700">
        Upload Logo
      </label>
      <input
        type="file"
        accept="image/*"
        onChange={handleLogoUpload}
        className="block w-full text-sm text-gray-500
        file:mr-4 file:py-2 file:px-4
        file:rounded-full file:border-0
        file:text-sm file:font-semibold
        file:bg-primary file:text-white

```

```

        hover:file:bg-indigo-700"
      />
      <button
        onClick={handleDeleteLogo}
        className="bg-red-600 text-white px-4 py-2 rounded text-sm
hover:bg-red-700 transition"
      >
        Remove
      </button>
    </div>
    <p className="text-xs text-gray-500 mt-2">Uploading a logo will
automatically update the site settings.</p>
  </div>

  <div className="border-t pt-6">
    <h4 className="text-md font-semibold mb-4">Site Details</h4>
    <form onSubmit={handleUpdateSettings} className="space-y-4">
      <div>
        <label className="block text-sm font-medium text-gray-700">Brand
Name</label>
        <input
          type="text"
          value={siteSettings.brandName}
          onChange={(e) => setSiteSettings({ ...siteSettings,
brandName: e.target.value })}
          className="mt-1 block w-full rounded-md border-gray-300
shadow-sm focus:border-indigo-500 focus:ring-indigo-500 sm:text-sm p-2 border"
          placeholder="EduPortal"
        />
      </div>
      <div>
        <label className="block text-sm font-medium text-gray-
700">Github Link</label>
        <input
          type="url"
          value={siteSettings.githubLink}
          onChange={(e) => setSiteSettings({ ...siteSettings,
githubLink: e.target.value })}
          className="mt-1 block w-full rounded-md border-gray-300
shadow-sm focus:border-indigo-500 focus:ring-indigo-500 sm:text-sm p-2 border"
          placeholder="https://github.com/..."
        />
      </div>
      <div>
        <label className="block text-sm font-medium text-gray-
700">LinkedIn Link</label>
        <input
          type="url"
          value={siteSettings.linkedinLink}
          onChange={(e) => setSiteSettings({ ...siteSettings,
linkedinLink: e.target.value })}
          className="mt-1 block w-full rounded-md border-gray-300

```

```

shadow-sm focus:border-indigo-500 focus:ring-indigo-500 sm:text-sm p-2 border"
        placeholder="https://linkedin.com/in/..."
      />
    </div>
    <div>
      <label className="block text-sm font-medium text-gray-
700">Copyright Text</label>
      <input
        type="text"
        value={siteSettings.copyrightText}
        onChange={(e) => setSiteSettings({ ...siteSettings,
copyrightText: e.target.value })}
        className="mt-1 block w-full rounded-md border-gray-300
shadow-sm focus:border-indigo-500 focus:ring-indigo-500 sm:text-sm p-2 border"
        placeholder="2026 Your Name"
      />
    </div>
    <button
      type="submit"
      className="bg-indigo-600 text-white px-4 py-2 rounded hover:bg-
indigo-700 transition"
    >
      Save Settings
    </button>
  </form>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-3 gap-6 mb-10">
  <div className="bg-white p-6 rounded-lg shadow border-l-4 border-blue-500">
    <h3 className="text-gray-500 text-sm font-medium">Total Students</h3>
    <p className="text-3xl font-bold text-gray-800">{stats.studentCount}</p>
  </div>
  <div className="bg-white p-6 rounded-lg shadow border-l-4 border-green-500">
    <h3 className="text-gray-500 text-sm font-medium">Total Teachers</h3>
    <p className="text-3xl font-bold text-gray-800">{stats.teacherCount}</p>
  </div>
  <div className="bg-white p-6 rounded-lg shadow border-l-4 border-purple-
500">
    <h3 className="text-gray-500 text-sm font-medium">Total Courses</h3>
    <p className="text-3xl font-bold text-gray-800">{stats.courseCount}</p>
  </div>
</div>

  {renderTable(pendingTeachers, 'teacher', 'Pending Teacher Approvals')}
  {renderTable(pendingStudents, 'student', 'Pending Student Approvals')}
</div>
);
};

export default AdminDashboard;

```

File: client/src/components/Footer.tsx

```
import React, { useEffect, useState } from 'react';
import { Github, Linkedin } from 'lucide-react';
import API from '../api';

interface SiteSettings {
  githubLink: string;
  linkedinLink: string;
  copyrightText: string;
  brandName?: string;
  logoUrl?: string;
}

const Footer: React.FC = () => {
  const [settings, setSettings] = useState<SiteSettings>({
    githubLink: 'https://github.com/Manish-Dark',
    linkedinLink: 'https://www.linkedin.com/in/manish-sharma-426039297',
    copyrightText: '2026 Manish Dark',
    brandName: 'EduPortal',
    logoUrl: ''
  });

  useEffect(() => {
    const fetchSettings = async () => {
      try {
        // Adjust URL if needed based on your API setup
        const res = await API.get('/site-settings');
        if (res.data) {
          setSettings(res.data);
        }
      } catch (error) {
        console.error('Failed to fetch site settings', error);
      }
    };

    fetchSettings();
  }, []);

  const logoUrl = settings.logoUrl ? settings.logoUrl : '';

  return (
    <footer className="bg-gray-900 text-white h-16 mt-auto w-full flex items-center">
      <div className="container mx-auto px-4 flex flex-col md:flex-row justify-between items-center h-full">
        <div className="flex items-center space-x-3">
          {logoUrl ? (
            <img src={logoUrl} alt="Logo" className="h-8 w-auto" />
          ) : (
            <span className="text-xl font-bold">{settings.brandName}</span>
          )}
        </div>
      </div>
    </footer>
  );
};
```

```

        <span className="text-sm font-light tracking-wider opacity-75">
          | &copy; {settings.copyrightText}
        </span>
      </div>

      <div className="flex space-x-6">
        <a
          href={settings.githubLink}
          target="_blank"
          rel="noopener noreferrer"
          className="hover:text-primary transition-colors duration-300"
          title="GitHub"
        >
          <Github size={24} />
        </a>
        <a
          href={settings.linkedinLink}
          target="_blank"
          rel="noopener noreferrer"
          className="hover:text-blue-500 transition-colors duration-300"
          title="LinkedIn"
        >
          <LinkedIn size={24} />
        </a>
      </div>
    </div>
  </footer>
);
};

export default Footer;

```

File: client/src/components/Navbar.tsx

```

import React from 'react';
import { Link, useNavigate } from 'react-router-dom';
import API, { BASE_URL } from '../api';
import { useAuth } from '../context/AuthContext';
import { LogOut, BookOpen, User } from 'lucide-react';
import { useState, useEffect } from 'react';

const Navbar: React.FC = () => {
  const { user, logout } = useAuth();
  const navigate = useNavigate();
  const [brandName, setBrandName] = useState('EduPortal');
  const [logoUrl, setLogoUrl] = useState('');

  useEffect(() => {
    const fetchSettings = async () => {
      try {

```

```

const res = await API.get('/site-settings');
if (res.data) {
  if (res.data.brandName) setBrandName(res.data.brandName);
  if (res.data.logoUrl) setLogoUrl(res.data.logoUrl);
}
} catch (error) {
  console.error('Failed to fetch site settings', error);
}
};
fetchSettings();
}, []));

return (
  <nav className="bg-white shadow-md">
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
      <div className="flex justify-between h-16">
        <div className="flex items-center cursor-pointer" onClick={() =>
navigate('/')}>
          {logoUrl ? (
            <img src={logoUrl.startsWith('http') ? logoUrl :
`${BASE_URL}${logoUrl}`} alt="Logo" className="h-10 w-auto mr-2" />
          ) : (
            <BookOpen className="h-8 w-8 text-primary" />
          )}
          <span className="ml-2 text-xl font-bold text-gray-800">{brandName}
</span>
        </div>
        <div className="flex items-center">
          {user ? (
            <>
              <Link to="/dashboard" className="text-gray-600 hover:text-
primary px-3 py-2 rounded-md text-sm font-medium">Dashboard</Link>
              <div className="flex items-center ml-4">
                <User className="h-5 w-5 text-gray-500 mr-1" />
                <span className="text-gray-700 mr-4 font-semibold">
{user.name} ({user.role || 'Student'})</span>
                <button onClick={logout} className="flex items-center
text-gray-600 hover:text-red-600 transition duration-150">
                  <LogOut className="h-5 w-5" />
                </button>
              </div>
            </>
          ) : (
            <>
              <Link to="/login" className="text-gray-600 hover:text-
primary px-3 py-2 rounded-md text-sm font-medium">Login</Link>
              <Link to="/register" className="bg-primary text-white px-4
py-2 rounded-md text-sm font-medium hover:bg-indigo-700 ml-3">Register</Link>
            </>
          )}
        </div>
      </div>
    </div>
  </div>

```

```

        </div>
      </nav>
    );
  };

  export default Navbar;

```

File: client/src/components/StudentDashboard.tsx

```

import React from 'react';
import { BASE_URL } from '../api';
import { Link } from 'react-router-dom';

const StudentDashboard: React.FC = () => {
  // Mock data or fetch enrolled courses removed
  const [materials, setMaterials] = React.useState<any[]>([]);

  const [user, setUser] = React.useState<any>(null);

  React.useEffect(() => {
    fetchMaterials();
    fetchProfile();
  }, []);

  const fetchMaterials = async () => {
    try {
      const profile = JSON.parse(localStorage.getItem('profile') || '{}');
      const token = profile.token;
      if (!token) return;

      const response = await fetch(`${BASE_URL}/api/materials`, {
        headers: { 'Authorization': `Bearer ${token}` }
      });
      if (response.ok) {
        setMaterials(await response.json());
      }
    } catch (error) {
      console.error('Error fetching materials:', error);
    }
  }

  const fetchProfile = async () => {
    try {
      const profile = JSON.parse(localStorage.getItem('profile') || '{}');
      const token = profile.token;
      if (!token) return;

      const response = await fetch(`${BASE_URL}/api/auth/me`, {
        headers: { 'Authorization': `Bearer ${token}` }
      });
      if (response.ok) {

```

```

        setUser(await response.json());
    }
} catch (error) {
    console.error('Error fetching profile:', error);
}
};

return (
    <div>
        {/* Profile Section */}
        {user && (
            <div className="bg-white p-6 rounded-lg shadow mb-8">
                <h2 className="text-2xl font-bold text-gray-800 mb-4">My Profile</h2>
                <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
                    <div>
                        <p className="text-gray-600">Name</p>
                        <p className="font-semibold">{user.name}</p>
                    </div>
                    <div>
                        <p className="text-gray-600">Email</p>
                        <p className="font-semibold">{user.email}</p>
                    </div>
                    <div>
                        <p className="text-gray-600">Course</p>
                        <p className="font-semibold">{user.course}</p>
                    </div>
                    {user.branch && (
                        <div>
                            <p className="text-gray-600">Branch</p>
                            <p className="font-semibold">{user.branch}</p>
                        </div>
                    )}
                    {user.fatherName && (
                        <div>
                            <p className="text-gray-600">Father's Name</p>
                            <p className="font-semibold">{user.fatherName}</p>
                        </div>
                    )}
                    {user.motherName && (
                        <div>
                            <p className="text-gray-600">Mother's Name</p>
                            <p className="font-semibold">{user.motherName}</p>
                        </div>
                    )}
                </div>
            </div>
        )}

        {/* Quiz History Section */}
        {user && user.progress && user.progress.length > 0 && (
            <div className="bg-white p-6 rounded-lg shadow mb-8">
                <h2 className="text-xl font-bold text-gray-800 mb-4">Quiz History</h2>

```

```

<div className="overflow-x-auto">
  <table className="min-w-full divide-y divide-gray-200">
    <thead className="bg-gray-50">
      <tr>
        <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">Quiz Title</th>
        <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">Score</th>
        <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">Total Questions</th>
        <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">Percentage</th>
      </tr>
    </thead>
    <tbody className="bg-white divide-y divide-gray-200">
      {user.progress.flatMap((p: any) => p.quizScores)
        .filter((qs: any) => qs.quizId) // Filter out deleted quizzes
        .map((qs: any, index: number) => {
          // Handle cases where quiz might be deleted (populated as null)

          const title = qs.quizId.title;
          const total = qs.quizId.questions?.length || 0;
          const percentage = total > 0 ? Math.round((qs.score / total) * 100) : 0;

          return (
            <tr key={index}>
              <td className="px-6 py-4 whitespace-nowrap text-sm font-medium text-gray-900">{title}</td>
              <td className="px-6 py-4 whitespace-nowrap text-sm text-gray-500">{qs.score}</td>
              <td className="px-6 py-4 whitespace-nowrap text-sm text-gray-500">{total}</td>
              <td className="px-6 py-4 whitespace-nowrap text-sm text-gray-500">
                <span className={`px-2 inline-flex text-xs leading-5 font-semibold rounded-full ${percentage >= 70 ? 'bg-green-100 text-green-800' : 'bg-yellow-100 text-yellow-800'} `}>
                  {percentage}%
                </span>
              </td>
            </tr>
          );
        })}
    </tbody>
  </table>
</div>
</div>

{ /* Study Materials Section */}

```

```

<div className="mb-8">
  <div className="flex justify-between items-center mb-4">
    <h2 className="text-2xl font-semibold text-gray-800">Study
Materials</h2>
    <button
      onClick={fetchMaterials}
      className="text-sm bg-blue-50 text-blue-600 px-3 py-1 rounded
hover:bg-blue-100 transition flex items-center"
    >
      <img alt="refresh icon" data-bbox="288 241 305 251"/> Refresh
    </button>
  </div>
  {materials.length === 0 ? (
    <div className="bg-white p-6 rounded-lg shadow text-center">
      <p className="text-gray-600">No study materials available for your
course/branch yet.</p>
    </div>
  ) : (
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
      {materials.map((material) => (
        <div key={material._id} className="bg-white p-6 rounded-lg
shadow hover:shadow-lg transition">
          <h3 className="text-lg font-bold text-gray-900 mb-2">
{material.title}</h3>
          <p className="text-gray-600 text-sm mb-4">
{material.description}</p>
          <div className="flex justify-between items-center text-xs
text-gray-500 mb-4">
            <span>{material.course} {material.branch ? ` -
${material.branch}` : ''}</span>
            <span>By: {material.uploadedBy?.name || 'Teacher'}</span>
          </div>
          <div className="flex space-x-2">
            {material.type === 'link' ? (
              <a
                href={material.linkUrl}
                target="_blank"
                rel="noopener noreferrer"
                className="flex-1 text-center bg-purple-100
text-purple-700 py-2 rounded hover:bg-purple-200 transition"
              >
                Open Link
              </a>
            ) : (
              <a
                href=
{`${BASE_URL}/api/materials/download/${material._id}?inline=true`}
                target="_blank"
                rel="noopener noreferrer"
                className="flex-1 text-center bg-indigo-100

```

```

text-indigo-700 py-2 rounded hover:bg-indigo-200 transition"
        >
            View
        </a>
        <a
            href=
{`${BASE_URL}/api/materials/download/${material._id}`}
            className="flex-1 text-center bg-green-100
text-green-700 py-2 rounded hover:bg-green-200 transition"
        >
            Download
        </a>
    </>
    )}
</div>
</div>
    )}
</div>

    { /* Quizzes Section */ }
    <QuizSection />

</div>
);
};

const QuizSection: React.FC = () => {
    const [quizzes, setQuizzes] = React.useState<any[]>([]);

    React.useEffect(() => {
        const fetchQuizzes = async () => {
            try {
                const profile = JSON.parse(localStorage.getItem('profile') || '{}');
                const token = profile.token;
                // Fetch all quizzes for now.
                // Future improvement: Filter by student's course/branch if available in
profile
                const response = await fetch(`${BASE_URL}/api/quizzes`, {
                    headers: { 'Authorization': `Bearer ${token}` }
                });
                if (response.ok) {
                    const data = await response.json();
                    setQuizzes(data);
                }
            } catch (error) {
                console.error('Error fetching quizzes:', error);
            }
        };
        fetchQuizzes();
    });
};

```

```

    }, []);

    if (quizzes.length === 0) return null;

    return (
      <div className="mb-8">
        <h2 className="text-2xl font-semibold text-gray-800 mb-4">Available Quizzes</h2>
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
          {quizzes.map((quiz) => (
            <div key={quiz._id} className="bg-white p-6 rounded-lg shadow
            hover:shadow-lg transition border-1-4 border-indigo-500">
              <div className="flex justify-between items-start mb-2">
                <h3 className="text-xl font-bold text-gray-900">{quiz.title}
              </h3>
              <span className="bg-indigo-100 text-indigo-800 text-xs font-
              semibold px-2.5 py-0.5 rounded">
                {quiz.questions?.length || 0} Qs
              </span>
            </div>
            <div className="text-sm text-gray-500 mb-4">
              {quiz.course && <span className="mr-2">🏠 {quiz.course}</span>
              {quiz.branch && <span>🌿 {quiz.branch}</span>}
            </div>
            <Link
              to={` /take-quiz/${quiz._id}`}
              className="block w-full text-center bg-indigo-600 text-white py-
              2 rounded hover:bg-indigo-700 transition font-medium"
            >
              Start Quiz
            </Link>
          </div>
        </div>
      </div>
    );
  };
};

export default StudentDashboard;

```

File: client/src/components/TakeQuiz.tsx

```

import React, { useState } from 'react';
import API from '../api';

interface QuizProps {
  quiz: any;
  onClose: () => void;
}

const TakeQuiz: React.FC<QuizProps> = ({ quiz, onClose }) => {
  const [answers, setAnswers] = useState<{ [key: number]: string }>({});

```

```

const [score, setScore] = useState<number | null>(null);

const handleSubmit = async () => {
  try {
    // Transform answers to array
    const answerArray = quiz.questions.map((_: any, index: number) => answers[index]
|| '');
    const { data } = await API.post('/quizzes/submit', { quizId: quiz._id, answers:
answerArray });
    setScore(data.score);
  } catch (error) {
    console.error(error);
  }
};

if (score !== null) {
  return (
    <div className="bg-white p-8 rounded-lg shadow-xl max-w-lg w-full mx-auto text-
center">
      <h2 className="text-2xl font-bold mb-4">Quiz Results</h2>
      <div className="text-5xl font-bold text-primary mb-2">{score} /
{quiz.questions.length}</div>
      <p className="text-gray-600 mb-6">Score: {Math.round((score /
quiz.questions.length) * 100)}%</p>
      <button onClick={onClose} className="bg-gray-800 text-white px-6 py-2
rounded-md hover:bg-gray-700">Close</button>
    </div>
  );
}

return (
  <div className="bg-white p-8 rounded-lg shadow-xl max-w-2xl w-full mx-auto max-h-
[90vh] overflow-y-auto">
    <div className="flex justify-between items-center mb-6">
      <h2 className="text-2xl font-bold">{quiz.title}</h2>
      <button onClick={onClose} className="text-gray-500 hover:text-gray-
700">X</button>
    </div>

    <div className="space-y-8">
      {quiz.questions.map((q: any, qIndex: number) => (
        <div key={qIndex}>
          <h4 className="font-semibold text-lg mb-3">{qIndex + 1}.
{q.questionText}</h4>
          <div className="space-y-2">
            {q.options.map((opt: string, oIndex: number) => (
              <label key={oIndex} className="flex items-center space-x-3
p-3 border rounded-md cursor-pointer hover:bg-gray-50">
                <input
                  type="radio"
                  name={`question-${qIndex}`}
                  value={opt}

```

```

    opt }}}

    checked={answers[qIndex] === opt}
    className="h-4 w-4 text-primary focus:ring-primary
border-gray-300"

    />
    <span className="text-gray-700">{opt}</span>
  </label>
)}}
</div>
</div>
)}}
</div>

<button
  onClick={handleSubmit}
  className="mt-8 w-full bg-primary text-white py-3 rounded-md font-bold
hover:bg-indigo-700 transition"
  >
    Submit Answers
  </button>
</div>
);
};

export default TakeQuiz;

```

File: client/src/components/TeacherDashboard.tsx

```
import React from 'react';
import { BASE_URL } from '../api';
import { Link } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
// Import API fetch if needed

const TeacherDashboard: React.FC = () => {
  const { user } = useAuth();
  // Mock for now removed

  if (!user.isApproved) {
    return (
      <div className="bg-yellow-50 border-1-4 border-yellow-400 p-4">
        <div className="flex">
          <div className="ml-3">
            <p className="text-sm text-yellow-700">
              Your account is pending approval by the administrator. You
cannot create courses yet.
            </p>
          </div>
        </div>
      </div>
    );
  }
}
```

```

    )
  }

  const [materialData, setMaterialData] = React.useState({ title: '', description: '',
course: 'B.Tech', branch: 'CSE', linkUrl: '' });
  const [files, setFiles] = React.useState<FileList | null>(null);
  const [uploadStatus, setUploadStatus] = React.useState('');

  const handleFileChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    if (e.target.files && e.target.files.length > 0) {
      setFiles(e.target.files);
      // Auto-set title from first filename if title is empty
      if (!materialData.title) {
        const name = e.target.files[0].name.replace(/\.^[^/]+\$/, "");
        setMaterialData(prev => ({ ...prev, title: name }));
      }
    }
  };

  // Old handleUpload removed, replaced by handleSubmit in the main render block logic

  const [activeOption, setActiveOption] = React.useState<'none' | 'upload' | 'link'>
('none');

  // ... (keep handleFileChange)

  // Unified submit handler or separate? Unified is fine.
  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    console.log('Handle Submit Called');
    console.log('Active Option:', activeOption);

    try {
      const profile = JSON.parse(localStorage.getItem('profile') || '{}');
      const token = profile.token;

      if (!token) {
        setUploadStatus('Authentication failed. Please login again.');
```

```

        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'Authorization': `Bearer ${token}`
        },
        body: JSON.stringify({
            title: materialData.title,
            description: materialData.description,
            course: materialData.course,
            branch: materialData.branch,
            linkUrl: materialData.linkUrl
        })
    });
} else {
    // Upload File
    if (!files || files.length === 0) {
        setUploadStatus('Please select at least one file');
        return;
    }

    if (files.length > 10) {
        setUploadStatus('You can upload a maximum of 10 files at once.');
```

return;

```

    }

    const formData = new FormData();
    formData.append('title', materialData.title);
    formData.append('description', materialData.description);
    formData.append('course', materialData.course);
    formData.append('branch', materialData.branch);
    formData.append('type', 'file');

    // Append all files
    for (let i = 0; i < files.length; i++) {
        formData.append('files', files[i]);
    }

    setUploadStatus('Uploading...');

    response = await fetch(`${BASE_URL}/api/materials/upload`, {
        method: 'POST',
        headers: {
            'Authorization': `Bearer ${token}`
        },
        body: formData
    });
}

if (response.ok) {
    setUploadStatus('Materials uploaded/linked successfully!');
    setMaterialData({ title: '', description: '', course: 'B.Tech', branch:

```

```

'CSE', linkUrl: '' });
    setFiles(null);

    // Reset file input value
    const fileInput = document.getElementById('file-upload') as
HTMLInputElement;
    if (fileInput) fileInput.value = '';

    fetchResources();
    setTimeout(() => { setActiveOption('none'); setUploadStatus(''); }, 2000);
  } else {
    const errorData = await response.json();
    setUploadStatus(errorData.message || 'Failed to upload material');
  }
} catch (error) {
  console.error('Error uploading material:', error);
  setUploadStatus('Error uploading material');
}
};

const [materials, setMaterials] = React.useState<any[]>([]);
const [quizzes, setQuizzes] = React.useState<any[]>([]);

React.useEffect(() => {
  fetchResources();
}, []);

const fetchResources = async () => {
  try {
    const profile = JSON.parse(localStorage.getItem('profile') || '{}');
    const token = profile.token;
    if (!token) return;

    const headers = { 'Authorization': `Bearer ${token}` };

    // Fetch Materials
    const matRes = await fetch(`${BASE_URL}/api/materials`, { headers });
    if (matRes.ok) setMaterials(await matRes.json());

    // Fetch Quizzes (Teacher sees all or we need a specific teacher route?
    // Current getQuizzes filters by student completion OR returns all.
    // Let's check getQuizzes logic: IF userRole is student checking completion.
    // IF teacher, it returns all (filtered by nothing if no queries).
    // We might want to filter by "my uploaded ones" ideally, but for now getting
all is fine per current backend logic.
    const quizRes = await fetch(`${BASE_URL}/api/quizzes`, { headers });
    if (quizRes.ok) setQuizzes(await quizRes.json());

  } catch (error) {
    console.error('Error fetching resources:', error);
  }
};

```

```

const handleDeleteMaterial = async (id: string) => {
  if (!window.confirm('Are you sure you want to delete this material?')) return;
  try {
    const profile = JSON.parse(localStorage.getItem('profile') || '{}');
    await fetch(`${BASE_URL}/api/materials/${id}`, {
      method: 'DELETE',
      headers: { 'Authorization': `Bearer ${profile.token}` }
    });
    fetchResources(); // Refresh list
  } catch (error) {
    console.error('Delete error:', error);
  }
};

const handleDeleteQuiz = async (id: string) => {
  if (!window.confirm('Are you sure you want to delete this quiz?')) return;
  try {
    const profile = JSON.parse(localStorage.getItem('profile') || '{}');
    await fetch(`${BASE_URL}/api/quizzes/${id}`, {
      method: 'DELETE',
      headers: { 'Authorization': `Bearer ${profile.token}` }
    });
    fetchResources(); // Refresh list
  } catch (error) {
    console.error('Delete error:', error);
  }
};

const [isResultsModalOpen, setIsResultsModalOpen] = React.useState(false);
const [selectedQuizResults, setSelectedQuizResults] = React.useState<any[]>([]);

const handleViewResults = async (quizId: string) => {
  try {
    const profile = JSON.parse(localStorage.getItem('profile') || '{}');
    const response = await fetch(`${BASE_URL}/api/quizzes/${quizId}/results`, {
      headers: { 'Authorization': `Bearer ${profile.token}` }
    });
    if (response.ok) {
      const data = await response.json();
      setSelectedQuizResults(data);
      setIsResultsModalOpen(true);
    } else {
      alert('Failed to fetch results');
    }
  } catch (error) {
    console.error('Error fetching results:', error);
    alert('Error fetching results');
  }
};

```

```

return (
  <div>
    <div className="flex justify-between items-center mb-6">
      <h2 className="text-2xl font-semibold text-gray-800">Teacher Dashboard</h2>
      {/* <Link to="/create-course" ... > removed create course button if not
needed, or keep it. User focused on materials/quiz. */}
    </div>

    {/* Selection Buttons */}
    {activeOption === 'none' && (
      <div className="grid grid-cols-1 md:grid-cols-3 gap-6 mb-8">
        <button
          onClick={() => setActiveOption('upload')}
          className="p-6 bg-blue-50 border border-blue-200 rounded-xl
hover:shadow-lg transition flex flex-col items-center justify-center text-center h-48"
        >
          <span className="text-4xl mb-4">📄</span>
          <h3 className="text-xl font-bold text-blue-800">Upload PDF</h3>
          <p className="text-blue-600 mt-2">Upload study materials, notes, or
assignments.</p>
        </button>

        <button
          onClick={() => setActiveOption('link')}
          className="p-6 bg-purple-50 border border-purple-200 rounded-xl
hover:shadow-lg transition flex flex-col items-center justify-center text-center h-48"
        >
          <span className="text-4xl mb-4">🔗</span>
          <h3 className="text-xl font-bold text-purple-800">Share Link</h3>
          <p className="text-purple-600 mt-2">Share YouTube videos, drive
links, or resources.</p>
        </button>

        <Link
          to="/create-quiz/general"
          className="p-6 bg-green-50 border border-green-200 rounded-xl
hover:shadow-lg transition flex flex-col items-center justify-center text-center h-48"
        >
          <span className="text-4xl mb-4">📝</span>
          <h3 className="text-xl font-bold text-green-800">Create Quiz</h3>
          <p className="text-green-600 mt-2">Create assessments for your
students.</p>
        </Link>
      </div>
    )}

    {/* Upload/Link Form */}
    {activeOption !== 'none' && (
      <div className="bg-white p-6 rounded-lg shadow-md mb-8 relative">
        <button
          onClick={() => setActiveOption('none')}

```

```

        className="absolute top-4 right-4 text-gray-400 hover:text-gray-600"
      >
        × Close
      </button>
      <h3 className="text-xl font-bold mb-4">
        {activeOption === 'upload' ? 'Upload Study Material (PDF)' : 'Share
External Link'}
      </h3>

      <form onSubmit={handleSubmit} className="space-y-4">
        {uploadStatus && <div className={`text-sm
${uploadStatus.includes('success') ? 'text-green-600' : 'text-red-600'}`}>{uploadStatus}
</div>}

        <div>
          <label className="block text-sm font-medium text-gray-
700">Title</label>

          <input
            type="text"
            className="mt-1 block w-full border border-gray-300 rounded-
md shadow-sm p-2"
            value={materialData.title}
            onChange={(e) => setMaterialData({ ...materialData, title:
e.target.value })}
            required
          />
        </div>

        {activeOption === 'link' && (
          <div>
            <label className="block text-sm font-medium text-gray-
700">Link URL (YouTube, Drive, etc.)</label>
            <input
              type="url"
              className="mt-1 block w-full border border-gray-300
rounded-md shadow-sm p-2"
              value={materialData.linkUrl || ''}
              onChange={(e) => setMaterialData({ ...materialData,
linkUrl: e.target.value })}
              required
              placeholder="https://..."
            />
          </div>
        )}

        <div>
          <label className="block text-sm font-medium text-gray-
700">Description</label>

          <textarea
            className="mt-1 block w-full border border-gray-300 rounded-
md shadow-sm p-2"
            value={materialData.description}

```

```

onChange={(e) => setMaterialData({ ...materialData,
description: e.target.value }}}
    />
  </div>

  <div className="grid grid-cols-2 gap-4">
    <div>
      <label className="block text-sm font-medium text-gray-
700">Target Course</label>

      <select
        className="mt-1 block w-full border border-gray-300
rounded-md shadow-sm p-2"
        value={materialData.course}
        onChange={(e) => setMaterialData({ ...materialData,
course: e.target.value }}}
      >
        <option value="B.Tech">B.Tech</option>
        <option value="M.Tech">M.Tech</option>
        <option value="BCA">BCA</option>
        <option value="MCA">MCA</option>
      </select>
    </div>
    <div>
      <label className="block text-sm font-medium text-gray-
700">Target Branch</label>

      <select
        className="mt-1 block w-full border border-gray-300
rounded-md shadow-sm p-2"
        value={materialData.branch}
        onChange={(e) => setMaterialData({ ...materialData,
branch: e.target.value }}}
      >
        <option value="CSE">CSE</option>
        <option value="CSD">CSD</option>
        <option value="AIML">AIML</option>
        <option value="Mechanical">Mechanical</option>
        <option value="Civil">Civil</option>
      </select>
    </div>
  </div>

  {activeOption === 'upload' && (
    <div>
      <label className="block text-sm font-medium text-gray-
700">PDF Files (Max 10)</label>

      <input
        id="file-upload"
        type="file"
        accept="application/pdf"
        multiple
        className="mt-1 block w-full"
        onChange={handleFileChange}

```

```

        required={activeOption === 'upload'}
      />
      {files && files.length > 0 && (
        <div className="mt-2 text-sm text-gray-600">
          Selected {files.length} file(s):
          <ul className="list-disc pl-5 mt-1">
            {Array.from(files).map((f, i) => (
              <li key={i}>{f.name}</li>
            ))}
          </ul>
        </div>
      )}
    </div>
  )}

  <div className="flex justify-end gap-3">
    <button type="button" onClick={() => setActiveOption('none')}
className="px-4 py-2 text-gray-600 hover:text-gray-800">
      Cancel
    </button>
    <button type="submit" className="bg-green-600 text-white px-4
py-2 rounded-md hover:bg-green-700">
      {activeOption === 'upload' ? 'Upload Material' : 'Share
Link'}
    </button>
  </div>
</form>
</div>
)}

{/* Manage Materials Section */}
<h3 className="text-xl font-bold mb-4 mt-8">Manage Materials</h3>
{materials.length === 0 ? (
  <p className="text-gray-500 mb-6">No materials uploaded yet.</p>
) : (
  <div className="bg-white rounded-lg shadow overflow-hidden mb-8">
    <table className="min-w-full divide-y divide-gray-200">
      <thead className="bg-gray-50">
        <tr>
          <th className="px-6 py-3 text-left text-xs font-medium text-
gray-500 uppercase tracking-wider">Title</th>
          <th className="px-6 py-3 text-left text-xs font-medium text-
gray-500 uppercase tracking-wider">Target</th>
          <th className="px-6 py-3 text-left text-xs font-medium text-
gray-500 uppercase tracking-wider">Type</th>
          <th className="px-6 py-3 text-right text-xs font-medium
text-gray-500 uppercase tracking-wider">Actions</th>
        </tr>
      </thead>
      <tbody className="bg-white divide-y divide-gray-200">
        {materials.map(m => (
          <tr key={m._id}>

```

```

        <td className="px-6 py-4 whitespace-nowrap">{m.title}
    </td>

        <td className="px-6 py-4 whitespace-nowrap text-sm text-
gray-500">{m.course} {m.branch}</td>

        <td className="px-6 py-4 whitespace-nowrap text-sm text-
gray-500 uppercase">{m.type}</td>

        <td className="px-6 py-4 whitespace-nowrap text-right
text-sm font-medium">

            <button onClick={() => handleDeleteMaterial(m._id)}
className="text-red-600 hover:text-red-900">Delete</button>

        </td>

    </tr>

    )}
</tbody>
</table>
</div>
)}

{/* Manage Quizzes Section */}
<h3 className="text-xl font-bold mb-4 mt-8">Manage Quizzes</h3>
{quizzes.length === 0 ? (
    <p className="text-gray-500 mb-6">No quizzes created yet.</p>
) : (
    <div className="bg-white rounded-lg shadow overflow-hidden mb-8">
        <table className="min-w-full divide-y divide-gray-200">
            <thead className="bg-gray-50">
                <tr>
                    <th className="px-6 py-3 text-left text-xs font-medium text-
gray-500 uppercase tracking-wider">Title</th>
                    <th className="px-6 py-3 text-left text-xs font-medium text-
gray-500 uppercase tracking-wider">Target</th>
                    <th className="px-6 py-3 text-left text-xs font-medium text-
gray-500 uppercase tracking-wider">Questions</th>
                    <th className="px-6 py-3 text-right text-xs font-medium
text-gray-500 uppercase tracking-wider">Actions</th>
                </tr>
            </thead>
            <tbody className="bg-white divide-y divide-gray-200">
                {quizzes.map(q => (
                    <tr key={q._id}>
                        <td className="px-6 py-4 whitespace-nowrap">{q.title}
                    </td>

                        <td className="px-6 py-4 whitespace-nowrap text-sm text-
gray-500">{q.course} {q.branch}</td>

                        <td className="px-6 py-4 whitespace-nowrap text-sm text-
gray-500">{q.questions?.length || 0}</td>

                        <td className="px-6 py-4 whitespace-nowrap text-right
text-sm font-medium">

                            <button onClick={() => handleViewResults(q._id)}
className="text-indigo-600 hover:text-indigo-900 mr-4">View Results</button>

                            <button onClick={() => handleDeleteQuiz(q._id)}
className="text-red-600 hover:text-red-900">Delete</button>

```

```

        </td>
      </tr>
    </tbody>
  </table>
</div>
)}

{/* Results Modal */}
{isResultsModalOpen && (
  <div className="fixed inset-0 bg-gray-600 bg-opacity-50 overflow-y-auto h-
full w-full flex items-center justify-center z-50">
    <div className="relative p-5 border w-full max-w-2xl shadow-lg rounded-
md bg-white">
      <div className="flex justify-between items-center mb-4">
        <h3 className="text-lg font-bold text-gray-900">Quiz
Results</h3>
        <button onClick={() => setIsResultsModalOpen(false)}
className="text-gray-500 hover:text-gray-700">X</button>
      </div>

      {selectedQuizResults.length === 0 ? (
        <p className="text-gray-500">No students have taken this quiz
yet.</p>
      ) : (
        <div className="overflow-x-auto">
          <table className="min-w-full divide-y divide-gray-200">
            <thead className="bg-gray-50">
              <tr>
                <th className="px-6 py-3 text-left text-xs font-
medium text-gray-500 uppercase tracking-wider">Name</th>
                <th className="px-6 py-3 text-left text-xs font-
medium text-gray-500 uppercase tracking-wider">Father's Name</th>
                <th className="px-6 py-3 text-left text-xs font-
medium text-gray-500 uppercase tracking-wider">Mother's Name</th>
                <th className="px-6 py-3 text-left text-xs font-
medium text-gray-500 uppercase tracking-wider">Email</th>
                <th className="px-6 py-3 text-left text-xs font-
medium text-gray-500 uppercase tracking-wider">Score</th>
              </tr>
            </thead>
            <tbody className="bg-white divide-y divide-gray-200">
              {selectedQuizResults.map((r: any, idx: number) => (
                <tr key={idx}>
                  <td className="px-6 py-4 whitespace-nowrap
text-sm font-medium text-gray-900">{r.name}</td>
                  <td className="px-6 py-4 whitespace-nowrap
text-sm text-gray-500">{r.fatherName || '-'}</td>
                  <td className="px-6 py-4 whitespace-nowrap
text-sm text-gray-500">{r.motherName || '-'}</td>
                  <td className="px-6 py-4 whitespace-nowrap
text-sm text-gray-500">{r.email}</td>

```

```

                                <td className="px-6 py-4 whitespace-nowrap
text-sm font-bold text-gray-900">{r.score}</td>
                                </tr>
                                )}}
                                </tbody>
                                </table>
                                </div>
                                )}
                                <div className="mt-4 flex justify-end">
                                    <button
                                        onClick={() => setIsResultsModalOpen(false)}
                                        className="px-4 py-2 bg-gray-200 text-gray-800 rounded
hover:bg-gray-300"
                                    >
                                        Close
                                    </button>
                                </div>
                            </div>
                        </div>
                    )
                }
            </div >
        );
    };
};

export default TeacherDashboard;

```

4. Backend Source Code (Node.js / Express)

4.1 Root

File: `server/index.js`

```

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const dotenv = require('dotenv');

const path = require('path');
dotenv.config({ path: path.join(__dirname, '.env') });

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
app.use(express.json());
app.use(cors());

// SIMPLE TEST ROUTE
app.get('/testdownload', (req, res) => {

```

```

    res.send('Test Download Route is Working!');
  });

app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});

// Import dependencies for download route
const StudyMaterial = require('./models/StudyMaterial');
// const path = require('path'); // Removed duplicate

const { downloadMaterial } = require('./controllers/material');

// Direct download route using controller (handles GridFS, Cloudinary, Local)
app.get('/api/download/:id', downloadMaterial);

// Database Connection
const seedAdmin = require('./seedAdmin');
const seedTeacher = require('./seedTeacher');

// Database Connection
const connectDB = require('./config/db');

// Database Connection
connectDB().then(() => {
  seedAdmin();
  seedTeacher();
});

const authRoutes = require('./routes/auth');

const courseRoutes = require('./routes/course');

const lessonRoutes = require('./routes/lesson');

const adminRoutes = require('./routes/admin');

const quizRoutes = require('./routes/quiz');

// Routes Configuration
// Routes Configuration
app.use('/api/auth', authRoutes);
app.use('/api/courses', courseRoutes);
app.use('/api/lessons', lessonRoutes);
app.use('/api/admin', adminRoutes);
app.use('/api/quizzes', quizRoutes);

const materialRoutes = require('./routes/material');
const siteSettingsRoutes = require('./routes/siteSettings');

// Direct download route to bypass potential router issues

```

```
// (Merged to top of file)

app.use('/api/materials', materialRoutes);
app.use('/api/site-settings', siteSettingsRoutes);
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

app.get('/', (req, res) => {
  res.send('API is running...');
});

// Sanity check route to verify server is reachable
app.get('/api/sanity', (req, res) => {
  res.send('Sanity check passed!');
});

app.get('/api/debug/materials', async (req, res) => {
  try {
    const materials = await StudyMaterial.find({});
    res.json(materials);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Start Server
app.use((req, res, next) => {
  console.log(`[404] Route not found: ${req.method} ${req.url}`);
  res.status(404).send(`Cannot GET (Logged) ${req.url}`);
});

app.listen(PORT, () => {
  console.log(`🚀 Server running on port ${PORT}`);
});

module.exports = app;
```

4.2 Config

File: server/config/cloudinary.js

```
const cloudinary = require('cloudinary').v2;
const { CloudinaryStorage } = require('multer-storage-cloudinary');
const dotenv = require('dotenv');

dotenv.config();
```

```

const cloudinaryConfigured = process.env.CLOUDINARY_CLOUD_NAME &&
process.env.CLOUDINARY_API_KEY && process.env.CLOUDINARY_API_SECRET;

if (cloudinaryConfigured) {
  cloudinary.config({
    cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
    api_key: process.env.CLOUDINARY_API_KEY,
    api_secret: process.env.CLOUDINARY_API_SECRET
  });
} else {
  console.warn('⚠️ Cloudinary credentials missing! Falling back to local disk storage.');
```

```

}

const storage = cloudinaryConfigured ? new CloudinaryStorage({
  cloudinary: cloudinary,
  params: async (req, file) => {
    const isPdf = file.mimetype === 'application/pdf';
    return {
      folder: 'online-learning-portal',
      resource_type: isPdf ? 'raw' : 'auto',
      public_id: file.originalname.split('.')[0].replace(/^[a-zA-Z0-9]/g, '') + '-' +
Date.now() + (isPdf ? '.pdf' : ''),
    };
  },
}) : require('multer').diskStorage({
  destination: (req, file, cb) => {
    const fs = require('fs');
    const path = require('path');
    const uploadDir = path.join(__dirname, '../uploads');
    if (!fs.existsSync(uploadDir)) {
      fs.mkdirSync(uploadDir, { recursive: true });
    }
    cb(null, uploadDir);
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, file.fieldname + '-' + uniqueSuffix + '.' +
file.originalname.split('.')[0].pop());
  }
});

module.exports = {
  cloudinary,
  storage
};

```

File: server/config/db.js

```

const mongoose = require('mongoose');
const dotenv = require('dotenv');
```

```

const path = require('path');

dotenv.config({ path: path.join(__dirname, '..', '.env') });

let connectionPromise;

const connectDB = () => {
  if (!connectionPromise) {
    connectionPromise = mongoose.connect(process.env.MONGO_URI)
      .then(m => {
        console.log('✅ MongoDB Connected via db.js');
        return m.connection.getClient(); // Resolve to MongoClient for GridFsStorage
      })
      .catch(err => {
        console.error('❌ MongoDB Connection Error:', err);
        process.exit(1);
      });
  }
  return connectionPromise;
};

module.exports = connectDB;

```

File: server/config/gridfs.js

```

const connectDB = require('./db');

console.log('GridFS Config: Initializing storage...');

const storage = new GridFsStorage({
  db: connectDB(),
  file: (req, file) => {
    // Use 'uploads' collection for files
    // and match original filename logic if possible, or just date-name
    const match = ['application/pdf', 'text/plain'];

    if (match.indexOf(file.mimetype) === -1) {
      // For other types (if any), maybe null? But we filter in router.
      return `${Date.now()}-${file.originalname}`;
    }

    return {
      bucketName: 'uploads', // This matches the collection name 'uploads.files'
      filename: `${Date.now()}-${file.originalname}`,
      contentType: file.mimetype // Explicitly save content type
    };
  }
});

module.exports = storage;

```

4.3 Models

File: server/models/Admin.js

```
const mongoose = require('mongoose');

const adminSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Admin', adminSchema);
```

File: server/models/Course.js

```
const mongoose = require('mongoose');

const courseSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: String,
  category: String,
  thumbnail: String, // URL
  instructor: { type: mongoose.Schema.Types.ObjectId, ref: 'Teacher', required: true },
  price: { type: Number, default: 0 }, // Free or Paid
  lessons: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Lesson' }],
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Course', courseSchema);
```

File: server/models/Lesson.js

```
const mongoose = require('mongoose');

const lessonSchema = new mongoose.Schema({
  title: { type: String, required: true },
  courseId: { type: mongoose.Schema.Types.ObjectId, ref: 'Course', required: true },
  videoUrl: { type: String, required: true },
  notes: String, // URL or text
  order: Number,
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Lesson', lessonSchema);
```

File: server/models/Quiz.js

```

const mongoose = require('mongoose');

const quizSchema = new mongoose.Schema({
  title: { type: String, required: true },
  courseId: { type: mongoose.Schema.Types.ObjectId, ref: 'Course' }, // Optional now
  course: { type: String }, // For string-based targeting (B.Tech, etc.)
  questions: [{
    questionText: { type: String, required: true },
    options: [{ type: String, required: true }],
    correctAnswer: { type: String, required: true } // Store the option text or index
  }],
  branch: { type: String }, // Optional, for specific branch targeting
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Quiz', quizSchema);

```

File: server/models/SiteSettings.js

```

const mongoose = require('mongoose');

const siteSettingsSchema = new mongoose.Schema({
  githubLink: {
    type: String,
    default: 'https://github.com/Manish-Dark'
  },
  linkedinLink: {
    type: String,
    default: 'https://www.linkedin.com/in/manish-sharma-426039297'
  },
  copyrightText: {
    type: String,
    default: '2026 Manish Dark'
  },
  brandName: {
    type: String,
    default: 'EduPortal'
  },
  logoUrl: {
    type: String,
    default: ''
  },
  backgroundUrl: {
    type: String,
    default: ''
  },
  lastUpdated: {
    type: Date,
    default: Date.now
  }
});

```

```

});

// Ensure only one document exists
siteSettingsSchema.statics.getSettings = async function () {
  let settings = await this.findOne();
  if (!settings) {
    settings = await this.create({});
  }
  return settings;
};

module.exports = mongoose.model('SiteSettings', siteSettingsSchema);

```

File: server/models/Student.js

```

const mongoose = require('mongoose');

const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  course: { type: String, required: true }, // e.g., B.Tech, M.Tech, BCA, MCA
  branch: { type: String }, // e.g., CSE, CSD, AIML (Required if course has branches)
  fatherName: { type: String },
  motherName: { type: String },
  enrolledCourses: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Course' }],
  progress: [{
    courseId: { type: mongoose.Schema.Types.ObjectId, ref: 'Course' },
    completedLessons: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Lesson' }],
    quizScores: [{
      quizId: { type: mongoose.Schema.Types.ObjectId, ref: 'Quiz' },
      score: Number
    }]
  }],
  isApproved: { type: Boolean, default: false },
  createdAt: { type: Date, default: Date.now }
}, { collection: 'students' });

module.exports = mongoose.model('Student', studentSchema);

```

File: server/models/StudyMaterial.js

```

const mongoose = require('mongoose');

const materialSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: String,
  type: { type: String, enum: ['file', 'link'], default: 'file' },
  fileUrl: {

```

```

    type: String,
    required: function () { return this.type === 'file'; }
  },
  linkUrl: {
    type: String,
    required: function () { return this.type === 'link'; }
  },
  course: { type: String, required: true },
  branch: { type: String }, // Optional, if applicable to specific branch
  uploadedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'Teacher', required: true },
  createdAt: { type: Date, default: Date.now }
}, { collection: 'materials' });

module.exports = mongoose.model('StudyMaterial', materialSchema);

```

File: server/models/Teacher.js

```

const mongoose = require('mongoose');

const teacherSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  course: { type: String, required: true }, // Department/Course they belong to
  isApproved: { type: Boolean, default: false },
  specialization: String,
  createdCourses: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Course' }],
  createdAt: { type: Date, default: Date.now }
}, { collection: 'teachers' });

module.exports = mongoose.model('Teacher', teacherSchema);

```

4.4 Controllers

File: server/controllers/admin.js

```

const Teacher = require('../models/Teacher');
const Student = require('../models/Student');
const Course = require('../models/Course');

const getStats = async (req, res) => {
  try {
    const studentCount = await Student.countDocuments();
    const teacherCount = await Teacher.countDocuments();
    const courseCount = await Course.countDocuments();
    res.status(200).json({ studentCount, teacherCount, courseCount });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

```

```

const { sendApprovalEmail, sendRejectionEmail } = require('../utils/email');

const getPendingTeachers = async (req, res) => {
  try {
    const teachers = await Teacher.find({ isApproved: false });
    res.status(200).json(teachers);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

const getPendingStudents = async (req, res) => {
  try {
    const students = await Student.find({ isApproved: false });
    res.status(200).json(students);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

const approveTeacher = async (req, res) => {
  const { id } = req.params;
  try {
    const teacher = await Teacher.findByIdAndUpdate(id, { isApproved: true }, { new:
true });
    if (teacher) {
      // Send email in next event loop tick to ensure no blocking
      setImmediate(() => {
        console.log(`Approving teacher: ${teacher.name}, Email: ${teacher.email}`);
        sendApprovalEmail(teacher.email, teacher.name)
          .then(() => console.log(`Email initiated for ${teacher.email}`))
          .catch(err => console.error('Email send failed', err));
      });
    }
    res.status(200).json(teacher);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

const rejectTeacher = async (req, res) => {
  const { id } = req.params;
  try {
    const teacher = await Teacher.findById(id);
    if (teacher) {
      setImmediate(() => {
        sendRejectionEmail(teacher.email, teacher.name).catch(err =>
console.error('Email send failed', err));
      });
      await Teacher.findByIdAndDelete(id); // Or keep with rejected flag
    }
  }
}

```

```

    res.status(200).json({ message: 'Teacher rejected and removed' });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

const approveStudent = async (req, res) => {
  const { id } = req.params;
  try {
    const student = await Student.findByIdAndUpdate(id, { isApproved: true }, { new:
true });
    if (student) {
      // Send email in next event loop tick
      setImmediate(() => {
        console.log(`Approving student: ${student.name}, Email: ${student.email}`);
        sendApprovalEmail(student.email, student.name)
          .then(() => console.log(`Email initiated for ${student.email}`))
          .catch(err => console.error('Email send failed', err));
      });
    }
    res.status(200).json(student);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

const rejectStudent = async (req, res) => {
  const { id } = req.params;
  try {
    const student = await Student.findById(id);
    if (student) {
      setImmediate(() => {
        sendRejectionEmail(student.email, student.name).catch(err =>
console.error('Email send failed', err));
      });
      await Student.findByIdAndDelete(id);
    }
    res.status(200).json({ message: 'Student rejected and removed' });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

module.exports = {
  getStats,
  getPendingTeachers,
  getPendingStudents,
  approveTeacher,
  rejectTeacher,
  approveStudent,

```

```
rejectStudent
};
```

File: server/controllers/auth.js

```
const Student = require('../models/Student');
const Teacher = require('../models/Teacher');
const Admin = require('../models/Admin');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

const register = async (req, res) => {
  try {
    const { name, email, password, role, course, branch, fatherName, motherName } =
req.body;
    let Model;
    if (role === 'student') Model = Student;
    else if (role === 'teacher') Model = Teacher;
    else if (role === 'admin') Model = Admin;
    else return res.status(400).json({ message: 'Invalid role' });

    const existingUser = await Model.findOne({ email });
    if (existingUser) return res.status(400).json({ message: 'User already exists' });

    // isApproved is FALSE for everyone by default (Student & Teacher), requiring Admin
    approval.
    // Admin is auto-approved if they manage to register via this route (though
    typically seeded).
    const isApproved = role === 'admin' ? true : false;

    const user = new Model({
      name,
      email,
      password: password,
      isApproved,
      course, // Save course
      branch, // Save branch (optional in schema, but passed if present)
      fatherName,
      motherName
    });

    await user.save();

    if (!isApproved) {
      return res.status(201).json({
        result: { name, email, role, isApproved },
        message: "Registration successful. Please wait for admin approval."
      });
    }

    const token = jwt.sign({ id: user._id, role }, process.env.JWT_SECRET, { expiresIn:
```

```

'1h' });

    res.status(201).json({ result: user, token });
  } catch (error) {
    res.status(500).json({ message: 'Something went wrong', error: error.message });
  }
};

const login = async (req, res) => {
  try {
    const { email, password, role } = req.body;

    if (!role) return res.status(400).json({ message: 'Role is required' });
    const normalizedRole = role.toLowerCase();

    let Model;
    if (normalizedRole === 'student') Model = Student;
    else if (normalizedRole === 'teacher') Model = Teacher;
    else if (normalizedRole === 'admin') Model = Admin;
    else return res.status(400).json({ message: 'Invalid role' }); // Or search all

    const user = await Model.findOne({ email });
    if (!user) return res.status(404).json({ message: 'User not found' });

    let isPasswordCorrect = false;

    // Direct comparison for ALL roles (Plain text)
    if (user.password === password) {
      isPasswordCorrect = true;
    } else {
      isPasswordCorrect = false;
    }

    if (!isPasswordCorrect) return res.status(400).json({ message: 'Invalid credentials'
});

    if (normalizedRole !== 'admin' && user.isApproved === false) {
      return res.status(403).json({ message: 'Account not approved yet. Please wait
for admin approval.' });
    }

    const token = jwt.sign({ id: user._id, role }, process.env.JWT_SECRET, { expiresIn:
'1h' });

    res.status(200).json({ result: user, token });
  } catch (error) {
    res.status(500).json({ message: 'Something went wrong', error: error.message });
  }
};

const getMe = async (req, res) => {
  try {

```

```

const userId = req.userId;
const role = req.userRole;

let user;
if (role === 'student') {
  user = await Student.findById(userId).populate({
    path: 'progress.quizScores.quizId',
    select: 'title questions'
  });
} else if (role === 'teacher') {
  user = await Teacher.findById(userId);
} else if (role === 'admin') {
  user = await Admin.findById(userId);
}

if (!user) return res.status(404).json({ message: 'User not found' });

// sanitize password
user.password = undefined;

res.status(200).json(user);
} catch (error) {
  res.status(500).json({ message: 'Error fetching profile', error: error.message });
}
};

module.exports = { register, login, getMe };

```

File: server/controllers/course.js

```

const Course = require('../models/Course');
const Teacher = require('../models/Teacher');

const getCourses = async (req, res) => {
  try {
    const courses = await Course.find().populate('instructor', 'name');
    res.status(200).json(courses);
  } catch (error) {
    res.status(404).json({ message: error.message });
  }
};

const createCourse = async (req, res) => {
  const course = req.body;

  if (!req.userId) return res.json({ message: 'Unauthenticated' });

  const newCourse = new Course({ ...course, instructor: req.userId, createdAt: new
Date().toISOString() });

  try {

```

```

    await newCourse.save();

    // Add course to teacher's createdCourses
    await Teacher.findByIdAndUpdate(req.userId, { $push: { createdCourses: newCourse._id
} });

    res.status(201).json(newCourse);
  } catch (error) {
    res.status(409).json({ message: error.message });
  }
};

const getCourse = async (req, res) => {
  const { id } = req.params;
  try {
    const course = await Course.findById(id).populate('instructor',
'name').populate('lessons');
    res.status(200).json(course);
  } catch (error) {
    res.status(404).json({ message: error.message });
  }
}

module.exports = { getCourses, createCourse, getCourse };

```

File: server/controllers/lesson.js

```

const Lesson = require('../models/Lesson');
const Course = require('../models/Course');

const createLesson = async (req, res) => {
  const lesson = req.body;

  // Basic validation
  if (!lesson.courseId || !lesson.title || !lesson.videoUrl) {
    return res.status(400).json({ message: 'Missing required fields' });
  }

  const newLesson = new Lesson({ ...lesson, createdAt: new Date().toISOString() });

  try {
    await newLesson.save();

    // Add lesson to course
    await Course.findByIdAndUpdate(lesson.courseId, { $push: { lessons: newLesson._id }
});

    res.status(201).json(newLesson);
  } catch (error) {
    res.status(409).json({ message: error.message });
  }
}

```

```

};

const getLessons = async (req, res) => {
  const { courseId } = req.params;
  try {
    const lessons = await Lesson.find({ courseId }).sort({ order: 1 });
    res.status(200).json(lessons);
  } catch (error) {
    res.status(404).json({ message: error.message });
  }
}

module.exports = { createLesson, getLessons };

```

File: server/controllers/material.js

```

const StudyMaterial = require('../models/StudyMaterial');
const Student = require('../models/Student');
const Teacher = require('../models/Teacher');
const path = require('path');
const fs = require('fs');
const mongoose = require('mongoose');

const uploadMaterial = async (req, res) => {
  try {
    console.log('Upload Request Body:', req.body);
    console.log('Upload Request Files (count):', req.files?.length);

    const { title, description, course, branch, type, linkUrl } = req.body;
    const files = req.files;

    // Validation based on type
    if (type === 'file' && (!files || files.length === 0)) {
      return res.status(400).json({ message: 'No files uploaded for file type material' });
    }
    if (type === 'link' && !linkUrl) {
      return res.status(400).json({ message: 'Link URL is required for link type material' });
    }

    if (type === 'link') {
      // Handle Link (Single)
      const materialData = {
        title,
        description,
        type: 'link',
        linkUrl,
        course,
        branch,
        uploadedBy: req.userId
      };

```

```

    };
    const material = new StudyMaterial(materialData);
    await material.save();
    return res.status(201).json({ message: 'Link added successfully', material });
  } else {
    // Handle Files (Multiple)
    const uploadedMaterials = [];

    for (const file of files) {
      // Determine Title: Use form title if single file, else use filename
      let materialTitle = title;
      if (files.length > 1) {
        // Use filename without extension
        materialTitle = path.parse(file.originalname).name;
      } else {
        // If title is empty, fallback to filename
        materialTitle = title || path.parse(file.originalname).name;
      }

      const materialData = {
        title: materialTitle,
        description,
        type: 'file',
        course,
        branch,
        uploadedBy: req.userId
      };

      if (file.buffer) {
        const bucket = new mongoose.mongo.GridFSBucket(mongoose.connection.db, {
bucketName: 'uploads' });
        const filename = `${Date.now()}-${file.originalname}`;

        const uploadStream = bucket.openUploadStream(filename, {
          contentType: file.mimetype,
          metadata: { contentType: file.mimetype }
        });

        await new Promise((resolve, reject) => {
          uploadStream.on('error', (error) => reject(error));
          uploadStream.on('finish', () => resolve());
          uploadStream.end(file.buffer);
        });

        materialData.fileUrl = filename;
      }

      const material = new StudyMaterial(materialData);
      await material.save();
      uploadedMaterials.push(material);
    }

    res.status(201).json({ message: 'Materials uploaded successfully', materials:

```

```

uploadedMaterials });
    }

    } catch (error) {
        console.error('Error uploading material:', error);
        res.status(500).json({ message: 'Error uploading material', error: error.message });
    }
};

const addLink = async (req, res) => {
    try {

        const { title, description, course, branch, linkUrl } = req.body;

        if (!linkUrl) {
            return res.status(400).json({ message: 'Link URL is required' });
        }

        const material = new StudyMaterial({
            title,
            description,
            type: 'link',
            linkUrl,
            course,
            branch,
            uploadedBy: req.userId
        });

        await material.save();
        res.status(201).json({ message: 'Link added successfully', material });
    } catch (error) {
        res.status(500).json({ message: 'Error adding link', error: error.message });
    }
};

const downloadMaterial = async (req, res) => {
    try {
        const material = await StudyMaterial.findById(req.params.id);

        if (!material) {
            return res.status(404).json({ message: 'Material not found' });
        }

        // Check if fileUrl is a remote URL (Cloudinary)
        if (material.fileUrl && material.fileUrl.startsWith('http')) {
            return res.redirect(material.fileUrl);
        }

        // Try to serve from GridFS
        const bucket = new mongoose.mongo.GridFSBucket(mongoose.connection.db, { bucketName:
'uploads' });

```

```

// Check if file exists in GridFS first to avoid stream errors on start
const cursor = bucket.find({ filename: material.fileUrl });
const files = await cursor.toArray();

if (files.length > 0) {
  // GridFS file found
  const file = files[0];

  // Determine Content-Type
  let contentType = file.contentType;
  if (!contentType && file.metadata && file.metadata.contentType) {
    contentType = file.metadata.contentType;
  }
  // Fallback to extension-based MIME type
  if (!contentType || contentType === 'application/octet-stream') {
    const ext = path.extname(material.fileUrl).toLowerCase();

    if (ext === '.pdf') contentType = 'application/pdf';
    else if (ext === '.txt') contentType = 'text/plain';
    else if (ext === '.jpg' || ext === '.jpeg') contentType = 'image/jpeg';
    else if (ext === '.png') contentType = 'image/png';
    else if (ext === '.doc' || ext === '.docx') contentType =
'application/msword';
  }

  res.set('Content-Type', contentType || 'application/octet-stream');

  const disposition = req.query.inline === 'true' ? 'inline' : 'attachment';
  const filename = material.title + (path.extname(material.fileUrl) || '.pdf');

  res.set('Content-Disposition', `${disposition}; filename="${filename}"`);

  const downloadStream = bucket.openDownloadStreamByName(material.fileUrl);
  downloadStream.pipe(res);
} else {
  // Not in GridFS, try local filesystem (fallback)
  const normalizedFileUrl = material.fileUrl.split('/').join(path.sep);
  const filePath = path.resolve(__dirname, '..', normalizedFileUrl);

  // Check if file exists locally
  if (fs.existsSync(filePath)) {
    const disposition = req.query.inline === 'true' ? 'inline' : 'attachment';

    // Determine Content-Type for local file
    let contentType = 'application/octet-stream';
    const ext = path.extname(material.fileUrl).toLowerCase();

    if (ext === '.pdf') contentType = 'application/pdf';

```

```

        else if (ext === '.txt') contentType = 'text/plain';
        else if (ext === '.jpg' || ext === '.jpeg') contentType = 'image/jpeg';
        else if (ext === '.png') contentType = 'image/png';

        res.set('Content-Type', contentType);
        res.download(filePath, material.title + path.extname(material.fileUrl), {
            headers: {
                'Content-Disposition': `${disposition}; filename="${material.title +
path.extname(material.fileUrl)}"`
            }
        });
    } else {
        if (!res.headersSent) {
            res.status(404).json({ message: 'File not found on server' });
        }
    }
}

} catch (error) {
    console.error('Error downloading material:', error);
    if (!res.headersSent) {
        res.status(500).json({ message: 'Error downloading material', error:
error.message });
    }
}
};

const getMaterials = async (req, res) => {
    try {
        const userId = req.userId;
        const userRole = req.userRole;

        let filter = {};

        if (userRole === 'student') {
            const student = await Student.findById(userId);
            if (!student) return res.status(404).json({ message: 'Student not found' });

            filter.course = student.course;
            // If student has a branch, filter by it. If material has no branch, it might be
general for the course?
            // Let's assume strict matching: Show materials that match student's branch OR
have no branch specified (common materials).
            if (student.branch) {
                filter.$or = [
                    { branch: student.branch },
                    { branch: null },
                    { branch: '' }
                ];
            } else {
                // For courses without branches (e.g. maybe BCA), show all or just those
with null branch

```

```

        // If student has no branch, show materials with no branch.
        filter.branch = { $in: [null, ''] }; // Simplified for now
    }

    } else if (userRole === 'teacher') {
        // Teacher sees what they uploaded, OR everything for their course?
        // "teacher are choose there corsee... give the teacher panle you give the area
        fro the uploading"
        // Let's let teachers see all materials for their course to avoid duplicates, or
        just their own.
        // Using "uploadedBy" to let them manage their own.
        filter.uploadedBy = userId;
    }

    const materials = await StudyMaterial.find(filter).populate('uploadedBy', 'name');
    res.status(200).json(materials);
} catch (error) {
    res.status(500).json({ message: 'Error fetching materials', error: error.message });
}
};

const deleteMaterial = async (req, res) => {
    try {
        const material = await StudyMaterial.findById(req.params.id);
        if (!material) return res.status(404).json({ message: 'Material not found' });

        // Authorization: Only uploader or admin can delete
        if (material.uploadedBy.toString() !== req.userId && req.userRole !== 'admin') {
            return res.status(403).json({ message: 'Not authorized to delete this material'
        });
    }

    // If it's URL (Cloudinary), allow deletion (we can't delete from Cloudinary easily
    without library import here, skipping for now as it's just a link in DB effectively)
    // If it's a file but not http...
    if (material.type === 'file' && material.fileUrl &&
    !material.fileUrl.startsWith('http')) {
        // Try GridFS delete
        const bucket = new mongoose.mongo.GridFSBucket(mongoose.connection.db, {
        bucketName: 'uploads' });
        const cursor = bucket.find({ filename: material.fileUrl });
        const files = await cursor.toArray();

        if (files.length > 0) {
            await bucket.delete(files[0]._id);
        } else {
            // Try local delete
            const filePath = path.resolve(__dirname, '..', material.fileUrl);
            if (fs.existsSync(filePath)) {
                fs.unlinkSync(filePath);
            }
        }
    }
}

```

```

    }

    await StudyMaterial.findByIdAndDelete(req.params.id);
    res.status(200).json({ message: 'Material deleted successfully' });
  } catch (error) {
    res.status(500).json({ message: 'Error deleting material', error: error.message });
  }
};

module.exports = { uploadMaterial, getMaterials, downloadMaterial, addLink, deleteMaterial
};

```

File: server/controllers/quiz.js

```

const Quiz = require('../models/Quiz');
const Student = require('../models/Student');

const createQuiz = async (req, res) => {
  const { title, courseId, questions, branch } = req.body;
  const newQuiz = new Quiz({
    title,
    courseId,
    questions,
    branch,
    createdAt: new Date().toISOString()
  });
  try {
    await newQuiz.save();
    res.status(201).json(newQuiz);
  } catch (error) {
    console.error('Create Quiz Error:', error);
    res.status(409).json({ message: error.message });
  }
}

const getQuizzes = async (req, res) => {
  try {
    const { courseId } = req.params;
    const { course, branch } = req.query;
    const userId = req.userId;
    const userRole = req.userRole;

    let query = {};

    if (courseId && courseId !== 'undefined' && courseId !== 'null') {
      if (courseId.match(/^[0-9a-fA-F]{24}$/)) {
        query.courseId = courseId;
      }
    }

    if (course) query.course = course;

```

```

    if (branch) query.branch = branch;

    let quizzes = await Quiz.find(query).sort({ createdAt: -1 });

    if (userRole === 'student') {
      const student = await Student.findById(userId);
      if (student) {
        const completedQuizIds = student.progress.reduce((acc, p) => {
          return acc.concat(p.quizScores.map(q => q.quizId.toString()));
        }, []);

        quizzes = quizzes.filter(q => !completedQuizIds.includes(q._id.toString()));
      }
    }

    res.status(200).json(quizzes);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

const getQuiz = async (req, res) => {
  const { id } = req.params;
  try {
    const quiz = await Quiz.findById(id);
    res.status(200).json(quiz);
  } catch (error) {
    res.status(404).json({ message: error.message });
  }
}

const submitQuiz = async (req, res) => {
  const { quizId, answers } = req.body;
  const userId = req.userId;

  try {
    const quiz = await Quiz.findById(quizId);
    if (!quiz) return res.status(404).json({ message: 'Quiz not found' });

    let score = 0;
    quiz.questions.forEach((q, index) => {
      if (answers[index] === q.correctAnswer) {
        score++;
      }
    });

    const student = await Student.findById(userId);
    if (!student) return res.status(404).json({ message: 'Student not found' });

    let targetProgressIndex = -1;

    // Try to match specific course progress

```

```

    if (quiz.courseId) {
      targetProgressIndex = student.progress.findIndex(p => p.courseId &&
p.courseId.toString() === quiz.courseId.toString());
    }

    // If match found, push there. Else push new entry.
    if (targetProgressIndex !== -1) {
      student.progress[targetProgressIndex].quizScores.push({ quizId, score });
    } else {
      student.progress.push({
        quizScores: [{ quizId, score }]
      });
    }

    await student.save();

    res.status(200).json({ score, total: quiz.questions.length });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

const deleteQuiz = async (req, res) => {
  try {
    const quiz = await Quiz.findById(req.params.id);
    if (!quiz) return res.status(404).json({ message: 'Quiz not found' });

    await Quiz.findByIdAndDelete(req.params.id);
    res.status(200).json({ message: 'Quiz deleted successfully' });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

const getQuizResults = async (req, res) => {
  try {
    const { id } = req.params; // quizId

    // Find students who have a score for this quizId
    const students = await Student.find({
      'progress.quizScores.quizId': id
    }).select('name email fatherName motherName progress');

    const results = students.map(student => {
      // Find the specific score entry for this quiz
      // Handling potential multiple entries (taking latest or highest? Let's take
latest for now)
      // Flatten quizScores from all progress entries
      const allScores = student.progress.flatMap(p => p.quizScores);
      const quizScoreEntry = allScores.filter(qs => qs.quizId.toString() ===
id).pop(); // Get last one

```

```

        return {
            studentId: student._id,
            name: student.name,
            email: student.email,
            fatherName: student.fatherName,
            motherName: student.motherName,
            score: quizScoreEntry ? quizScoreEntry.score : 0
        };
    });

    res.status(200).json(results);
} catch (error) {
    res.status(500).json({ message: error.message });
}
};

module.exports = { createQuiz, getQuizzes, getQuiz, submitQuiz, deleteQuiz, getQuizResults
};

```

4.5 Routes

File: `server/routes/admin.js`

```

const express = require('express');
const router = express.Router();
const multer = require('multer');
const path = require('path');
const fs = require('fs');
const { auth } = require('../middleware/auth');
const {
    getStats,
    getPendingTeachers,
    getPendingStudents,
    approveTeacher,
    rejectTeacher,
    approveStudent,
    rejectStudent
} = require('../controllers/admin');
const { sendApprovalEmail, sendRejectionEmail, sendEmail } = require('../utils/email');

// Middleware to check if user is admin (hardcoded email or role check)
const adminCheck = (req, res, next) => {
    // In a real app, check for specific admin role or email
    // For this demo, we'll assume any 'teacher' role can access admin to simplify testing,
    // OR we can make a specific Admin model.
    // The prompt says "Admin Module (Lightweight)".
    // Let's assume a hardcoded admin email or a role 'admin' if we had one.
    // For now, let's allow teachers to act as admins to bootstrap, or better,
    // let's check for a specific flag or just allow "teacher" for now as the prompt implies
    // separate Admin module.
    // I'll add a check that they passed a secret header or just reuse auth.

```

```

    // Let's assume the user logged in as 'admin' role (which we haven't strictly enforced
in registration).
    // I'll add 'admin' role support in registration/login to make this clean.

    if (req.userRole === 'admin') {
        next();
    } else {
        return res.status(403).json({ message: 'Admin access denied' });
    }
};

// Import existing Cloudinary storage configuration
const { storage } = require('../config/cloudinary');
const upload = multer({ storage: storage });

// Routes

// Upload background image
router.post('/upload-background', auth, adminCheck, upload.single('image'), (req, res) => {
    if (!req.file) {
        return res.status(400).json({ message: 'No file uploaded' });
    }

    // We update the site settings background field or return path
    // Wait, background image isn't in SiteSettings schema? Let's assume it is or we just
return the URL
    // Looking at AdminDashboard.tsx, it just calls this and then relies on
`/uploads/landing-bg.jpg`.
    // This implies we need to actually save the background URL in the SiteSettings, or
return it and let the frontend save it.

    // But since the frontend component `AdminDashboard.tsx` relies on hardcoded
`/uploads/landing-bg.jpg`
    // Let's actually update the SiteSettings object here.
    res.json({ message: 'Background image updated successfully', filePath: req.file.path });
});

// Upload Logo
router.post('/upload-logo', auth, adminCheck, upload.single('image'), (req, res) => {
    if (!req.file) {
        return res.status(400).json({ message: 'No file uploaded' });
    }
    res.json({ message: 'Logo updated successfully', filePath: req.file.path });
});

const SiteSettings = require('../models/SiteSettings');

// Helper function to extract public_id from Cloudinary URL
const extractPublicId = (url) => {
    if (!url || !url.includes('cloudinary.com')) return null;
    const parts = url.split('/');
    // Get the part after /upload/ (usually version/id.ext or just id.ext)

```

```

const uploadIndex = parts.findIndex(p => p === 'upload');
if (uploadIndex === -1) return null;

// The public_id includes the folder.
// e.g. .../upload/v1234/online-learning-portal/img-name.jpg
const fileWithFolder = parts.slice(uploadIndex + 2).join('/');
// Remove the extension
return fileWithFolder.split('.')[0];
};

// Delete background image
router.delete('/background', auth, adminCheck, async (req, res) => {
  try {
    const settings = await SiteSettings.getSettings();
    const url = settings.backgroundUrl;

    if (url) {
      const publicId = extractPublicId(url);
      if (publicId) {
        const { cloudinary } = require('../config/cloudinary');
        await cloudinary.uploader.destroy(publicId);
      }
      // Clear in DB
      settings.backgroundUrl = '';
      await settings.save();
    }
    res.json({ message: 'Background image removed successfully' });
  } catch (error) {
    console.error('Error deleting background image:', error);
    res.status(500).json({ message: 'Failed to complete deletion' });
  }
});

// Delete logo
router.delete('/logo', auth, adminCheck, async (req, res) => {
  try {
    const settings = await SiteSettings.getSettings();
    const url = settings.logoUrl;

    if (url) {
      const publicId = extractPublicId(url);
      if (publicId) {
        const { cloudinary } = require('../config/cloudinary');
        await cloudinary.uploader.destroy(publicId);
      }
      // Clear in DB
      settings.logoUrl = '';
      await settings.save();
    }
    res.json({ message: 'Logo removed successfully' });
  } catch (error) {
    console.error('Error deleting logo:', error);
  }
});

```

```

        res.status(500).json({ message: 'Failed to complete deletion' });
    }
});

router.get('/stats', auth, adminCheck, getStats);

// Teachers
router.get('/teachers/pending', auth, adminCheck, getPendingTeachers);
router.put('/teachers/:id/approve', auth, adminCheck, approveTeacher);
router.put('/teachers/:id/reject', auth, adminCheck, rejectTeacher);

// Students
router.get('/students/pending', auth, adminCheck, getPendingStudents);
router.put('/students/:id/approve', auth, adminCheck, approveStudent);
router.put('/students/:id/reject', auth, adminCheck, rejectStudent);

module.exports = router;

```

File: server/routes/auth.js

```

const express = require('express');
const { register, login } = require('../controllers/auth');

const router = express.Router();

const { auth } = require('../middleware/auth');

router.post('/register', register);
router.post('/login', login);
router.get('/me', auth, require('../controllers/auth').getMe);

module.exports = router;

```

File: server/routes/course.js

```

const express = require('express');
const { getCourses, createCourse, getCourse } = require('../controllers/course');
const { auth, teacherLimit } = require('../middleware/auth');

const router = express.Router();

router.get('/', getCourses);
router.get('/:id', getCourse);
router.post('/', auth, teacherLimit, createCourse);

module.exports = router;

```

File: server/routes/lesson.js

```

const express = require('express');
const { createLesson, getLessons } = require('../controllers/lesson');
const { auth, teacherLimit } = require('../middleware/auth');

const router = express.Router();

router.get('/:courseId', getLessons);
router.post('/', auth, teacherLimit, createLesson);

module.exports = router;

```

File: server/routes/material.js

```

const express = require('express');
const router = express.Router();
const multer = require('multer');
const path = require('path');

console.log('Material Routes File Loaded'); // Debug log

router.use((req, res, next) => {
  console.log(`[Materials Router] ${req.method} ${req.url}`);
  next();
});

const { uploadMaterial, getMaterials, downloadMaterial, addLink } =
require('../controllers/material');
const { auth, teacherLimit } = require('../middleware/auth');

// const { storage } = require('../config/cloudinary');
const storage = multer.memoryStorage();
console.log('Storage Engine Type:', storage.constructor.name);

const upload = multer({
  storage: storage,
  fileFilter: (req, file, cb) => {
    if (file.mimetype === 'application/pdf' || file.mimetype === 'text/plain') {
      cb(null, true);
    } else {
      cb(new Error('Only PDFs and Text files are allowed'), false);
    }
  }
});

console.log('addLink Type:', typeof addLink);

router.post('/upload', auth, teacherLimit, upload.array('files', 10), uploadMaterial);
router.post('/link', auth, teacherLimit, addLink); // New route for links (JSON body)
router.get('/', auth, getMaterials);
router.delete('/:id', auth, teacherLimit,

```

```

require('../controllers/material').deleteMaterial);
router.get('/download/:id', downloadMaterial);
router.get('/test', (req, res) => res.json({ message: 'Material Routes Working' }));

router.use((req, res) => {
  console.log(`[Materials Router] Unhandled Request: ${req.method} ${req.url}`);
  res.status(404).json({ message: 'Material route not handled' });
});

// Print registered routes for debugging
router.stack.forEach(r => {
  if (r.route && r.route.path) {
    console.log(`[Materials Route Registered]
${Object.keys(r.route.methods).join(',').toUpperCase()} ${r.route.path}`);
  }
});

module.exports = router;

```

File: server/routes/quiz.js

```

const express = require('express');
const { createQuiz, getQuizzes, getQuiz, submitQuiz } = require('../controllers/quiz');
const { auth, teacherLimit } = require('../middleware/auth');

const router = express.Router();

router.post('/', auth, teacherLimit, createQuiz);
router.get('/', auth, getQuizzes); // New route for general fetching with query params
router.get('/course/:courseId', auth, getQuizzes); // Existing route
router.get('/:id', auth, getQuiz);
router.post('/submit', auth, submitQuiz);
router.delete('/:id', auth, teacherLimit, require('../controllers/quiz').deleteQuiz);
router.get('/:id/results', auth, teacherLimit,
require('../controllers/quiz').getQuizResults);

module.exports = router;

```

File: server/routes/siteSettings.js

```

const express = require('express');
const router = express.Router();
const SiteSettings = require('../models/SiteSettings');
const { auth } = require('../middleware/auth');

// Get Site Settings (Public)
router.get('/', async (req, res) => {
  try {
    const settings = await SiteSettings.getSettings();

```

```

    res.json(settings);
  } catch (error) {
    res.status(500).json({ message: 'Error fetching site settings', error: error.message
  });
}
});

// Update Site Settings (Admin only - using auth middleware + role check)
// Assuming auth middleware adds user to req.user
router.put('/', auth, async (req, res) => {
  try {
    // Simple admin check
    if (req.userRole !== 'admin') {
      return res.status(403).json({ message: 'Access denied. Admin only.' });
    }

    const { githubLink, linkedinLink, copyrightText, brandName, logoUrl, backgroundUrl }
= req.body;

    // Find and update or create if not exists (though getSettings ensures existence)
    let settings = await SiteSettings.findOne();
    if (!settings) {
      settings = new SiteSettings();
    }

    if (githubLink !== undefined) settings.githubLink = githubLink;
    if (linkedinLink !== undefined) settings.linkedinLink = linkedinLink;
    if (copyrightText !== undefined) settings.copyrightText = copyrightText;
    if (brandName !== undefined) settings.brandName = brandName;
    if (logoUrl !== undefined) settings.logoUrl = logoUrl;
    if (backgroundUrl !== undefined) settings.backgroundUrl = backgroundUrl;
    settings.lastUpdated = Date.now();

    await settings.save();
    res.json(settings);
  } catch (error) {
    res.status(500).json({ message: 'Error updating site settings', error: error.message
  });
}
});

module.exports = router;

```

4.6 Middleware

File: server/middleware/auth.js

```

const jwt = require('jsonwebtoken');

const auth = (req, res, next) => {
  try {

```

```
    console.log('Auth Middleware Header:', req.headers.authorization);
    if (!req.headers.authorization) return res.status(401).json({ message: 'No
authorization header' });

    const token = req.headers.authorization.split(' ')[1];
    const isCustomAuth = token.length < 500;

    let decodedData;

    if (token && isCustomAuth) {
        decodedData = jwt.verify(token, process.env.JWT_SECRET);
        req.userId = decodedData?.id;
        req.userRole = decodedData?.role;
        console.log('Decoded Token:', decodedData);
    } else {
        decodedData = jwt.decode(token);
        req.userId = decodedData?.sub;
    }

    next();
} catch (error) {
    console.log('Auth Middleware Error:', error);
    res.status(401).json({ message: 'Unauthenticated' });
}
};

const teacherLimit = (req, res, next) => {
    if (req.userRole === 'teacher' || req.userRole === 'admin') {
        next();
    } else {
        res.status(403).json({ message: 'Access denied. Teachers only.' });
    }
}

module.exports = { auth, teacherLimit };
```